Descripción de la Arquitectura del Software (SAD)

Sistema Integral de Gestión de Eventos

Equipo Hexacore

Versión: 2.0 – ADR-16

Revisión técnica: 02/11/2026

Autores:
Daniel Cristancho
Samuel Emperador
Sebastián Sánchez
Diego Coronado

Materia: Arquitectura de Software

Historial de Cambios

Fecha	Descripción del cambio	Persona(s)
25/08/2026	Creación de la versión 1.0 del documento, consolidando el trabajo ya elaborado en *ArchitecturalProposal.tex* (atributos de calidad y propuesta arquitectónica), *C4Diagrams.tex* (los seis diagramas C4), *Submission/DistribucionCasosUso.pdf* (los 32 casos de uso) y *BitacoraArquitectonica.md* (decisiones, alternativas y riesgos). Se añade además un primer Árbol de Utilidad (ASR) y los ADR de las decisiones fundacionales, que hasta ahora solo existían en la bitácora.	Equipo Hexacore (vía asistente)
27/08/2026	Versión 1.1: se cierran los puntos pendientes de la v1.0 — diagrama de clases UML del modelo de dominio, vista de componentes de los contenedores Personal, Eventos/Emergencias y Parqueaderos, ambientes de desarrollo y pruebas en la vista física, y modelo de datos detallado por microservicio.	Equipo Hexacore (vía asistente)
28/08/2026	Versión 1.2: durante la construcción del prototipo de la app móvil (Entrega 2) se decide unificar **App Móvil del Cliente** y **App Móvil del Personal** en un único contenedor **App Móvil**, que identifica el rol del usuario en el login y navega al conjunto de pantallas correspondiente (ver ADR-07). Se actualiza la vista de contenedores (de cuatro a tres interfaces desplegables) y ADR-02/ADR-05 en consecuencia.	Equipo Hexacore (vía asistente)

Fecha	Descripción del cambio	Persona(s)
22/09/2026	Versión 1.3: se añade la sección **Tácticas arquitectónicas por atributo de calidad** (§15), que dice cómo se realiza en el código cada atributo priorizado y con qué medición se respalda. Cubre los cinco que la entrega exige demostrar en vivo: desempeño, disponibilidad, **seguridad**, **consistencia** y desplegabilidad. Las cifras de la columna «Evidencia medida» provienen de ejecutar los guiones de `pruebas/` contra el sistema desplegado, no de estimaciones de diseño.	Equipo Hexacore (vía asistente)
18/10/2026	Versión 1.4: se registra **ADR-12**, que revisa la granularidad fijada en ADR-01 y adopta una **arquitectura basada en servicios** — tres servicios de dominio en lugar de seis: un núcleo que agrupa Administración y Pedidos como módulos, y los servicios replicables de Entradas / Mercado Secundario y de Eventos / Emergencias. Incluye el criterio de extracción con el que se decide qué dominio queda como servicio propio, y la reformulación de la métrica de RNF-13 que la decisión obliga. ADR-01 queda marcado como superado.	Equipo Hexacore (vía asistente)

Fecha	Descripción del cambio	Persona(s)
23/10/2026	Versión 1.5: se registra **ADR-14**, que revisa el criterio de extracción de ADR-12 —de dos condiciones a cuatro— y adopta una **arquitectura híbrida entre SOA y microservicios**: siete microservicios con base propia y réplicas elásticas, y siete servicios de dominio sobre una base central con un esquema por servicio. Incorpora además la Saga orquestada de CU-001, la réplica de lectura para Finanzas y el criterio de redundancia sin escalado para los servicios de dominio. ADR-12 queda marcado como superado. **Las vistas de contenedores y de despliegue (§9, §12) siguen describiendo ADR-12 a propósito**: es la arquitectura que está desplegada. ADR-14 está tomada y no ejecutada; la distancia entre ambas, y el orden para recorrerla, están en `Documentation/Work/MIGRACION-ADR-14.md`.	Equipo Hexacore (vía asistente)
23/10/2026	Versión 1.6: el documento pasa a describir **únicamente** la arquitectura de ADR-14. Se reemplazan la vista de contenedores (§9) y la vista física (§12) por las de los catorce servicios y el clúster con autoescalado, y el modelo de datos (§13) por los dos regímenes de almacenamiento —base propia por microservicio y base central con un esquema por servicio de dominio—. ADR-12 se conserva como registro superado, que es lo que corresponde a un ADR. Las secciones afectadas llevan una nota de **estado de implementación**: lo desplegado sigue siendo ADR-12 sobre Docker Compose. Se mantiene por exactitud, no como arquitectura alternativa.	Equipo Hexacore (vía asistente)

Fecha	**Descripción del cambio**	**Persona(s)**
30/10/2026	Versión 1.7: se registra **ADR-15**, que corrige cuatro relaciones de la vista de contenedores —el gateway valida el JWT con la clave pública y exige el turno de la Cola virtual; Acceso QR deja de llamar a Ventas y valida contra su propia copia de los QR; el *webhook* de la pasarela entra por el gateway— y fija lo que el diagrama dejaba abierto: tecnologías, réplicas mínimas, Transactional Outbox, el evento `EntradaActualizada` que invalida el QR anterior en una reventa y `EventoCancelado`. El conteo de contenedores queda en 27.	Equipo Hexacore (vía asistente)

Tabla de Contenidos

Historial de Cambios **1**

1 Introducción **6**

2 Visión general de los requisitos funcionales **6**

3 Requisitos No Funcionales (RNF) **7**

4 Modelo de dominio conceptual (requisitos) **10**

5 Stakeholders e intereses **13**

6 Requisitos Arquitectónicamente Significativos (ASR) **13**

7 Restricciones **19**

8 Contexto y alcance vigente **19**

9 Vista de contenedores **20**

10 Vista de componentes **21**

11 Vista de procesos **22**

12 Vista física **23**

13 Modelo de datos vigente **24**

14 Registros de Decisiones Arquitectónicas (ADR) **24**

15 Tácticas y mediciones históricas por atributo de calidad **43**

15.1 Desempeño 43

15.2 Disponibilidad 44

15.3 Seguridad 46

15.4 Consistencia 48

15.5 Desplegabilidad 50

16 Riesgos técnicos **51**

17 Glosario **52**

18 Referencias **53**

1. Introducción

El **Sistema Integral de Gestión de Eventos** centraliza en una sola plataforma los procesos que hoy suelen estar dispersos en la organización de un evento masivo (conciertos, festivales, partidos deportivos): venta de entradas y su mercado secundario, gestión de personal y turnos, parqueaderos, pedidos y establecimientos de alimentos, logística y operación del evento, y la administración general (usuarios, roles, recintos, proveedores, pagos y reportes). El problema central que resuelve es la **ausencia de un sistema unificado** que coordine estos procesos de forma consistente, segura y escalable, evitando información duplicada, errores humanos y falta de trazabilidad, especialmente crítico cuando hay miles de asistentes concurrentes durante la apertura de venta o el ingreso al recinto.

Los principales objetivos de negocio del sistema son: (1) permitir la venta y reventa segura de entradas sin duplicar propietarios ni sobrevender localidades; (2) mantener la disponibilidad del servicio durante los picos de tráfico propios de la apertura de venta, el ingreso masivo y las emergencias; (3) dar trazabilidad y auditoría a las operaciones financieras y administrativas; y (4) ofrecer una experiencia adecuada a cada tipo de usuario (cliente, personal operativo, administración) sin que ninguna interfaz duplique lógica de negocio.

Los atributos de calidad de mayor prioridad para la arquitectura son **consistencia**, **disponibilidad**, **seguridad** y **rendimiento**, seguidos de **tiempo real**, **escalabilidad** y **trazabilidad**; con prioridad baja-media se consideran **usabilidad**, **mantenibilidad** y **portabilidad**. La sección 6 detalla estos atributos en formato de Árbol de Utilidad.

Este documento sigue la estructura estándar *arc42* y se organiza así: primero se describen los requisitos funcionales y el modelo de dominio (secciones 2-4), luego los interesados y los requisitos arquitectónicamente significativos (secciones 5-6), después las restricciones del proyecto (sección 7) y las vistas arquitectónicas C4 (contexto, contenedores, componentes, procesos y física; secciones 8-12), y finalmente el modelo de datos, los registros de decisión (ADR), los riesgos técnicos, el glosario y las referencias.

2. Visión general de los requisitos funcionales

El sistema define **32 casos de uso** (CU-001 a CU-032), cada uno con un identificador único usado a lo largo de todo el documento. La especificación detallada de cada uno (actores, flujo principal, flujos alternativos y excepciones) está en la hoja de cálculo *Submission/CU_ eventos_ completo.xlsx*; la Tabla 2 resume los bloques temáticos en los que se agrupan.

Bloque temático	Casos de uso	Alcance funcional
Boletería / Compra de entradas	CU-001 – CU-005	Consulta de eventos, compra, validación de QR en el ingreso, cancelaciones/devoluciones y promociones.

Bloque temático	Casos de uso	Alcance funcional
Mercado secundario y personal para eventos	CU-006 – CU-010	Reventa segura de entradas (caso complejo), administración y asignación de personal, cambios de turno, registro de asistencia y gestión de evacuación ante emergencias (caso complejo).
Pedidos y establecimientos de alimentos	CU-011 – CU-015	Pedido de alimentos durante el evento, administración de menú/inventario, preparación y entrega validada por QR, y gestión de los establecimientos.
Logística y operación del evento	CU-016 – CU-020	Planificación logística, asignación de personal operativo, gestión de turno/asistencia en campo, monitoreo del evento y gestión de incidentes.
Parqueaderos	CU-021 – CU-025	Reserva, ingreso/salida de vehículos, asignación de espacios, cobro y control de ocupación.
Administración general (CRUD)	CU-026 – CU-032	Gestión de eventos, cuentas de usuario, roles y permisos, recintos y zonas, proveedores, pagos/conciliación financiera, y reportes y analítica.

Tabla 2: Bloques temáticos de los 32 casos de uso del sistema.

Los casos de uso están repartidos entre los cuatro integrantes del equipo (8 por persona); esa distribución, con nombre y descripción completa de cada CU, está documentada en *Submission/DistribucionCasosUso.pdf*. Dos casos de uso se destacan como los de mayor complejidad arquitectónica y sirven de hilo conductor en las vistas de este documento: **CU-006 (Gestión del Mercado Secundario de Entradas)** y **CU-010 (Gestionar Evacuación ante Emergencias)**.

3. Requisitos No Funcionales (RNF)

Mientras los casos de uso de la sección anterior describen *qué* hace el sistema, los requisitos no funcionales describen *con qué calidad* debe hacerlo. Cada RNF de la Tabla 3 se deriva de uno de los atributos de calidad priorizados en *ArchitecturalProposal.tex* y se expresa con una **métrica y un umbral verificable**, de modo que pueda comprobarse objetivamente si se cumple o no — un requisito redactado como "el sistema debe ser rápido" no es verificable y por lo tanto no se admite en esta tabla.

Los RNF son la entrada directa de las dos secciones siguientes: los **ASR** (sección 6) seleccionan cuáles de estos requisitos tienen impacto arquitectónico real, y el análisis de tácticas de *ArchitecturalProposal.tex* justifica con qué mecanismo se satisface cada uno.

Nota sobre los umbrales. Los valores numéricos son estimaciones razonadas del equipo a partir de los escenarios del proyecto, no mediciones sobre un sistema en producción (que aún

no existe). Los umbrales de RNF-01 y RNF-02 se respaldan con las pruebas de concepto del proyecto (véase *App/PoCs/*); el resto se validará conforme se implemente cada dominio.

ID	Requisito	Atributo	Métrica y umbral verificable
RNF-01	Ninguna entrada publicada en el mercado secundario puede venderse a dos compradores (CU-006).	Consistencia	Ventas duplicadas bajo concurrencia: **0%** con 50 intentos de compra simultáneos sobre la misma publicación.
RNF-02	Al transferirse una entrada, el QR anterior queda invalidado antes de emitir el nuevo (CU-006).	Consistencia	Códigos QR antiguos aceptados en validación tras una transferencia: **0**.
RNF-03	El sistema sigue atendiendo solicitudes ante la caída de una instancia de cualquier microservicio (CU-001–005, CU-002).	Disponibilidad	Disponibilidad mensual del API Gateway $\geq$ **99,9%** (máx. 43 min de indisponibilidad al mes).
RNF-04	Ante la caída de una instancia, el sistema se recupera sin intervención manual.	Disponibilidad	Tiempo hasta volver a responder el 100% de las solicitudes: $\leq$ **30 s**.
RNF-05	El sistema nunca almacena datos completos de tarjetas de pago (CU-001, CU-011, CU-024).	Seguridad	Campos de tarjeta persistidos en cualquier base de datos propia: **0**; el cobro se delega íntegramente a la pasarela.
RNF-06	Toda petición a un microservicio pasa autenticada y autorizada por rol desde el API Gateway.	Seguridad	Endpoints de negocio alcanzables sin token válido: **0%** sobre el total de rutas expuestas.
RNF-07	La consulta de disponibilidad de entradas responde con baja latencia durante la apertura de venta (CU-001).	Rendimiento	Latencia **p95 $\leq$ 500 ms** con 2.000 usuarios concurrentes.
RNF-08	La validación del QR en el ingreso no genera filas perceptibles (CU-002, CU-005).	Rendimiento	Latencia **p95 $\leq$ 300 ms** por validación.
RNF-09	El aforo y la ocupación se reflejan casi de inmediato tras un ingreso o salida (CU-019, CU-025).	Tiempo real	Retardo entre el evento y su reflejo en el tablero de monitoreo: $\leq$ **2 s**.
RNF-10	El servicio de Entradas escala horizontalmente sin degradar los demás dominios (CU-001, CU-006).	Escalabilidad	Al duplicar sus instancias, el throughput crece $\geq$ **1,7**$\times$ (eficiencia $\geq$ 85%) sin afectar la latencia de otros servicios.

ID	Requisito	Atributo	Métrica y umbral verificable
RNF-11	Toda operación financiera queda auditada (CU-006, CU-031).	Trazabilidad	Operaciones con registro completo (actor, marca de tiempo, id. de transacción, resultado): **100%**; retención $\geq$ **1 año**.
RNF-12	Un cliente sin capacitación previa completa la compra de una entrada (CU-001).	Usabilidad	$\geq$ **80%** de éxito en prueba con 5 usuarios, en $\leq$ **5 pasos** desde el evento hasta la confirmación.
RNF-13	Un cambio en las reglas de un dominio no obliga a modificar otro microservicio.	Mantenibilidad	Microservicios que deben modificarse y redesplegarse ante un cambio típico de negocio: **1** (más el contrato de API solo si este cambia).
RNF-14	Toda funcionalidad del rol Cliente está disponible desde web y desde móvil sobre el mismo contrato.	Portabilidad	CU del rol Cliente cubiertos por ambas interfaces consumiendo la misma API: **100%**, con **0** reglas de negocio duplicadas en el cliente.
RNF-15	Una versión nueva de un microservicio se despliega sin interrumpir el servicio.	Desplegabilidad	Solicitudes fallidas durante el despliegue: **0**; ventana de despliegue $\leq$ **10 min** y reversión $\leq$ **5 min**.
RNF-16	La pasarela de pagos puede sustituirse por otro proveedor sin tocar la lógica de negocio (CU-001, CU-024).	Integrabilidad	Componentes a modificar para el cambio de proveedor: **1** (solo el adaptador), **0** cambios en el servicio de Entradas.
RNF-17	Al activarse una evacuación, todo el personal en campo recibe su instrucción (CU-010).	Seguridad física (Safety)	Dispositivos que reciben la alerta: $\geq$ **99%** en $\leq$ **10 s** desde la activación.
RNF-18	Cada microservicio puede probarse de forma aislada, sin levantar los demás.	Comprobabilidad	Cobertura de pruebas de la lógica de dominio $\geq$ **70%** y **100%** de las dependencias externas sustituibles por dobles de prueba.

Tabla 3: Requisitos no funcionales, con métrica y umbral verificable por atributo de calidad.

4. Modelo de dominio conceptual (requisitos)

La Tabla 4 resume los conceptos del dominio identificados a partir de los 32 casos de uso. La Figura 1 presenta el diagrama de clases UML correspondiente (versión simplificada, con atributos clave y multiplicidades de las asociaciones principales); un modelo con navegabilidad, tipos de dato exactos y restricciones se refinará junto con el modelo entidad-relación de cada microservicio (sección 13) durante el prototipo.

Concepto	**Explicación**
Evento	Actividad que un organizador crea, con fecha, recinto, localidades, precios y aforo. Su **categoría** (concierto, teatro, deportes, festival, gastronomía) es un atributo del evento y es el criterio con el que el cliente filtra la cartelera.
Entrada	Boleto asociado a una localidad de un evento, con un código QR único que identifica al propietario actual.
Publicación de reventa	Oferta de una entrada ya adquirida, puesta en venta por su propietario en el mercado secundario.
Recinto / Zona	Lugar físico donde se realiza el evento y sus subdivisiones (zonas, localidades), cada una con capacidad y mapa de distribución.
Usuario / Cuenta	Persona registrada en el sistema (cliente, empleado, organizador o administrador), con credenciales y un conjunto de roles asignados.
Rol / Permiso	Conjunto de acciones autorizadas sobre un módulo del sistema, asignado a una cuenta de usuario.
Personal / Empleado	Persona que trabaja en la operación de un evento, con un rol operativo (seguridad, logística, alimentos, etc.).
Turno / Asignación	Periodo de trabajo de un empleado en un evento, con horario y zona asignada.
Registro de asistencia	Marca de ingreso y salida de un empleado durante un turno, usada para calcular horas trabajadas.
Parqueadero / Espacio	Zona de estacionamiento del evento y cada unidad de espacio reservable dentro de ella.
Pedido	Solicitud de productos de alimentos hecha por un cliente a un establecimiento durante un evento.
Establecimiento / Producto	Punto de venta de alimentos dentro del evento y los productos de su menú.
Pago	Transacción monetaria asociada a una compra, reventa, pedido o servicio de parqueadero, procesada mediante la pasarela de pagos externa.
Proveedor	Tercero externo (catering, sonido, seguridad privada, etc.) contratado para la operación de un evento.

Concepto	**Explicación**
Incidente / Protocolo de evacuación	Evento anómalo registrado durante la operación (desde uno menor hasta una emergencia que activa un protocolo de evacuación).
Reporte	Consulta agregada (ventas, ocupación, ingresos, personal, incidentes) usada para la toma de decisiones.

Tabla 4: Conceptos del dominio identificados a partir de los 32 casos de uso.

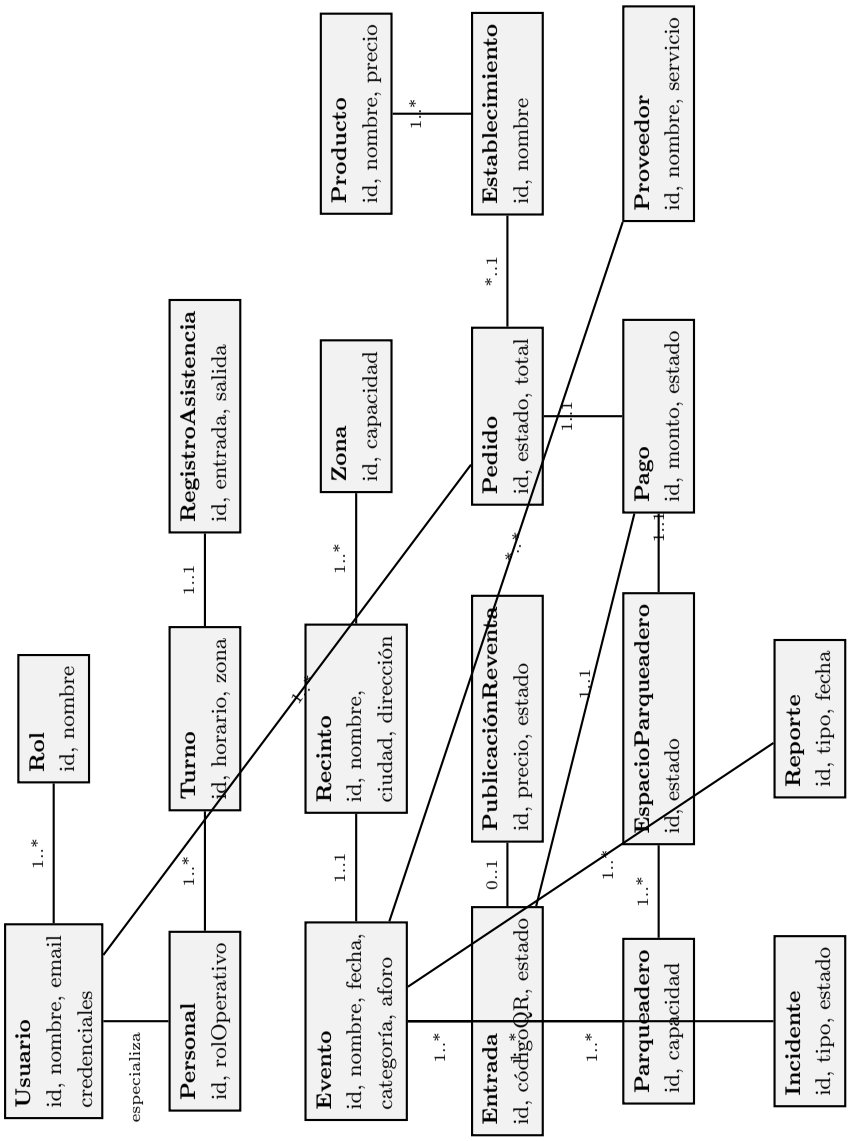


Figura 1: Diagrama de clases UML — modelo de dominio (versión simplificada).

5. Stakeholders e intereses

Rol/Nombre	**Información de Contacto**	**Intereses / expectativas**
Cliente	Rol dentro del sistema (se autentica con su cuenta)	Comprar y revender entradas de forma confiable, consultar sus pedidos y parqueadero, y recibir su QR sin fallas en el ingreso.
Organizador	Rol dentro del sistema	Crear y planificar eventos, asignar personal y proveedores, y monitorear la operación en tiempo real.
Empleado / Personal operativo	Rol dentro del sistema	Consultar sus turnos y asignaciones, solicitar cambios de turno y registrar su asistencia sin fricción.
Administrador	Rol dentro del sistema	Gestionar usuarios, roles, recintos y configuración general con control de acceso auditable.
Supervisor de emergencia	Rol dentro del sistema	Activar y coordinar protocolos de evacuación con información confiable y actualizada en tiempo real.
Proveedor externo	Gestionado por el organizador (módulo Proveedores)	Ser asignado correctamente a los eventos que contrata y recibir la información logística necesaria.
Equipo Hexacore (Daniel Cristancho, Samuel Emperador, Sebastián Sánchez, Diego Coronado)	`semperadorcontreras@gmail.com` (contacto del equipo)	Diseñar y sustentar una arquitectura que satisfaga los atributos de calidad priorizados y cumpla los requisitos del curso.
Profesor / Evaluador del curso	Curso de Arquitectura de Software	Verificar que el SAD cumpla la plantilla exigida y que las decisiones arquitectónicas estén justificadas.

6. Requisitos Arquitectónicamente Significativos (ASR)

Objetivos conservados. ADR-16 documenta concesiones propuestas respecto a los ASR de despliegue/datos siguientes; no cambia los requisitos de negocio ni afirma cumplimiento.

La Tabla 6 presenta el Árbol de Utilidad del proyecto: los escenarios de calidad que tienen impacto arquitectónico real, derivados de los atributos priorizados en *ArchitecturalProposal.tex* y de los requisitos no funcionales de la sección 3. La columna **RNF** indica qué requisito verificable

pone a prueba cada escenario; entre los dos elementos hay una diferencia de propósito: el RNF fija el umbral (*cuánto*), el ASR describe la situación concreta que lo tensiona (*cuándo y bajo qué condiciones*).

Cada escenario se califica con el par **(Importancia, Dificultad)** en escala Alta/Media/Baja, como pide el método del Árbol de Utilidad:

- **Importancia**: qué tanto afecta al negocio que ese escenario *no* se cumpla. Es la importancia *del escenario*, que no siempre coincide con la prioridad global de su atributo — por ejemplo, la Seguridad física está priorizada como Media en el conjunto del sistema, pero el escenario ASR-13 (fallar al notificar una evacuación) tiene importancia Alta porque compromete la integridad de las personas.
- **Dificultad**: qué tan difícil es técnicamente satisfacerlo con la arquitectura propuesta.

Los escenarios calificados **(Alta, Alta)** — ASR-01, ASR-04 y ASR-13 — son los que concentran el riesgo arquitectónico del proyecto y, por lo tanto, los candidatos naturales a respaldarse con una prueba de concepto medida en vez de solo con razonamiento.

Tabla 6: Árbol de Utilidad — escenarios de calidad con su par (Importancia, Dificultad) y el RNF que verifican.

ID ASR	**Título**	**Atributo**	**Sub-atributo**	**CU(s)**	**RNF**	**Escenario**	**Importancia**	**Dificultad**
ASR-01	Bloqueo de doble venta en reventa	Consistencia	Concurrencia de datos	CU-006	RNF-01, RNF-02	Dos compradores intentan adquirir simultáneamente la misma entrada publicada en reventa; el sistema garantiza que solo uno complete la compra y el otro reciba de inmediato un mensaje de no disponibilidad.	Alta	Alta
ASR-02	Disponibilidad durante el ingreso masivo	Disponibilidad	Tolerancia a fallos	CU-002, CU-021, CU-022	RNF-03, RNF-04	Ante la caída de una instancia del servicio de validación de QR durante el ingreso al recinto, el balanceador redirige el tráfico a instancias sanas sin interrupción perceptible para el asistente.	Alta	Media
ASR-03	Protección de datos de pago y credenciales	Seguridad	Confidencialidad	CU-001, CU-024, CU-031	RNF-05, RNF-06	Un intento de acceso no autorizado a datos de pago o credenciales es rechazado por el API Gateway y queda registrado para auditoría.	Alta	Media

Continúa en la página siguiente...

Tabla 6 — continuación

ID ASR	Título	Atributo	Sub-atributo	CU(s)	RNF	Escenario	Importancia	Dificultad
ASR-04	Respuesta bajo carga en apertura de venta	Rendimiento	Tiempo de respuesta	CU-001, CU-005	RNF-07, RNF-08	Durante la apertura de venta de un evento masivo, miles de usuarios consultan disponibilidad y compran de forma simultánea; la mayoría de las solicitudes se resuelve en tiempos cortos y estables.	Alta	Alta
ASR-05	Actualización de aforo y zonas en tiempo real	Tiempo real	Actualización de estado	CU-010, CU-019	RNF-09	El aforo y el estado de las zonas se reflejan en los tableros de monitoreo con baja latencia tras el registro de un ingreso o un incidente.	Media	Alta
ASR-06	Escalado independiente del servicio de Entradas	Escalabilidad	Escalabilidad horizontal	CU-001, CU-006	RNF-10	Ante un incremento súbito de tráfico en el servicio de Entradas durante una preventa, aumentan sus instancias sin afectar el rendimiento de los demás dominios.	Media	Media
ASR-07	Auditoría de transferencias y pagos	Trazabilidad	Auditabilidad	CU-006, CU-031	RNF-11	Cada transferencia de entrada, pago o reembolso queda registrada con usuario, fecha y resultado, consultable en un reporte de auditoría.	Media	Baja
ASR-08	Interfaces especializadas por rol	Usabilidad	Adecuación a la tarea	Todos los CU (por interfaz)	RNF-12	Un usuario nuevo de cada rol completa su tarea principal en su interfaz correspondiente sin capacitación previa.	Media	Baja

Continúa en la página siguiente...

Tabla 6 — continuación

ID ASR	Título	Atributo	Sub-atributo	CU(s)	RNF	Escenario	Importancia	Dificultad
ASR-09	Evolución independiente de dominios	Mantenibilidad	Modificabilidad	Todos los CU (por microservicio)	RNF-13	Un cambio en las reglas de negocio de Parqueaderos se implementa y despliega sin requerir cambios ni redespliegue del servicio de Entradas.	Media	Media
ASR-10	Consumo uniforme vía API	Portabilidad	Independencia de plataforma	Todos los CU	RNF-14	Una misma funcionalidad de backend se consume igual desde el portal web y desde la app móvil, sin duplicar lógica de negocio.	Media	Baja
ASR-11	Sustitución del proveedor de pagos sin reescribir el dominio	Integrabilidad	Adaptación de interfaces	CU-001, CU-011, CU-024	RNF-16	La organización cambia de pasarela de pagos por costos; el equipo reemplaza el adaptador de pagos y los servicios de Entradas, Parqueaderos y Pedidos siguen funcionando sin modificar su lógica de negocio.	Media	Media
ASR-12	Actualización de un servicio con el evento en curso	Desplegabilidad	Despliegue sin interrupción	Todos los CU (por microservicio)	RNF-15	Se detecta un defecto en el servicio de Pedidos durante un evento activo; se despliega la corrección sin detener la venta ni el ingreso de asistentes, y con posibilidad de revertir si la versión nueva falla.	Media	Media

Continúa en la página siguiente...

Tabla 6 — continuación

ID ASR	Título	Atributo	Sub-atributo	CU(s)	RNF	Escenario	Importancia	Dificultad
ASR-13	Notificación garantizada de evacuación	Seguridad física (Safety)	Respuesta ante emergencia	CU-010	RNF-17	El supervisor activa el protocolo de evacuación; todo el personal en campo recibe su instrucción específica en segundos, incluso si el proveedor de notificaciones responde con lentitud o una instancia del servicio falla.	Alta	Alta
ASR-14	Prueba aislada de cada microservicio	Comprobabilidad	Aislamiento de dependencias	Todos los CU (por microservicio)	RNF-18	Un integrante prueba la lógica de su dominio sin levantar los otros cinco microservicios ni depender de la pasarela de pagos real, sustituyéndolos por dobles de prueba.	Media	Baja

7. Restricciones

Restricción	Categoría	Descripción
Arquitectura de microservicios con base de datos independiente por dominio	Técnica	Decisión fundacional ya tomada por el equipo (ver ADR-01); cualquier solución a un ASR debe respetar esta separación por dominio.
Soporte de picos de miles de usuarios concurrentes	Técnica	El sistema debe soportar la concurrencia propia de la apertura de venta y el ingreso masivo al recinto sin degradar el servicio.
Equipo de 4 integrantes, 8 CU por persona	Organizacional	La distribución de trabajo (Daniel Cristancho, Samuel Emperador, Sebastián Sánchez, Diego Coronado) condiciona qué componentes prototipa cada integrante para Entrega 1 y Entrega 2.
Documento basado en el estándar arc42	Convencional	La plantilla y estructura de este documento las exige el curso; no pueden omitirse secciones.
Fechas y formato de entrega del curso	Convencional	Entrega 1 (SAD v1) y Entrega 2 (prototipo + SAD v2) deben respetar las fechas y reglas de entrega definidas por el curso.

8. Contexto y alcance vigente

Código migrado y probado en rama de arquitectura; corte habitual pendiente. Se conservan los 32 CU y su reparto del SRS, sin afirmar que todos estén implementados. El estado funcional del plan distingue base existente, simulación de interfaz y pendientes.

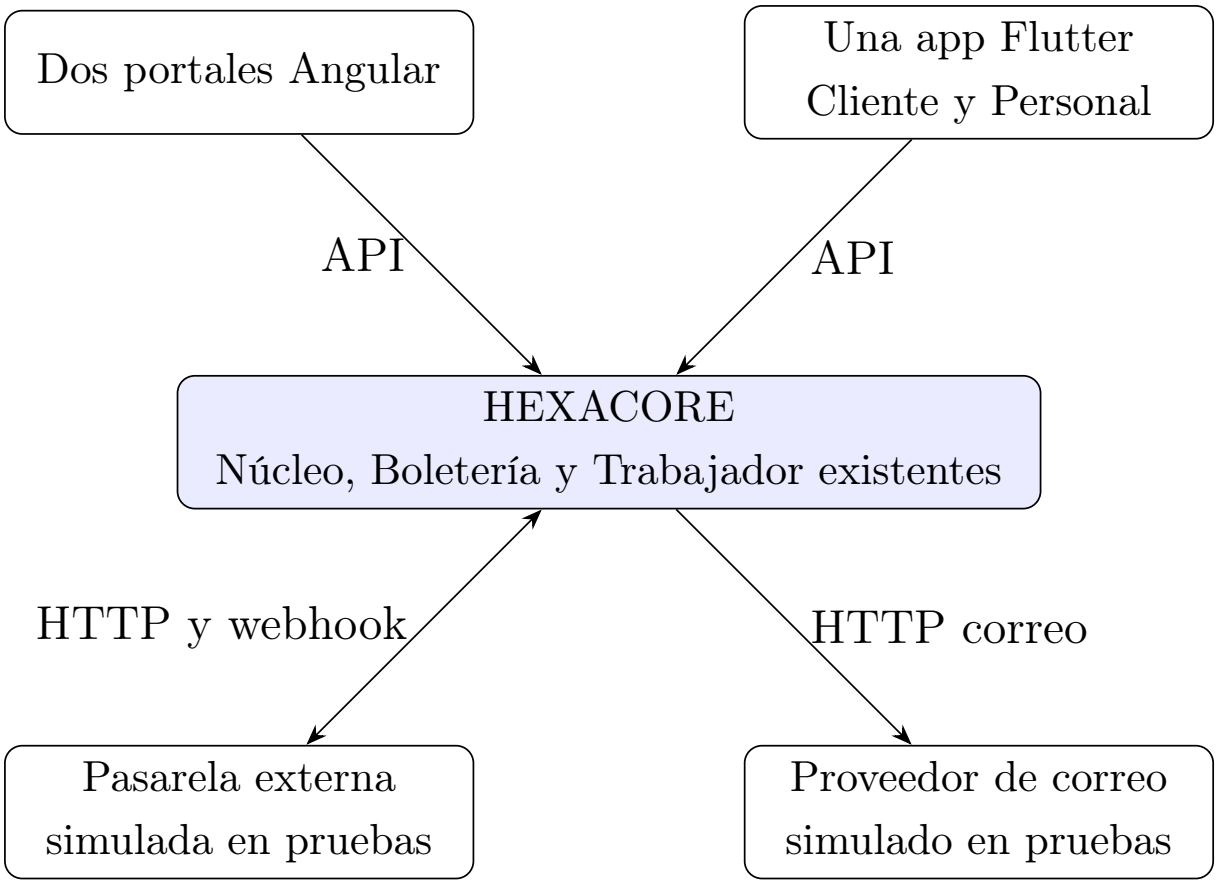


Figura 2: Contexto implementado: dos portales, una app móvil y proveedores simulados en pruebas.

9. Vista de contenedores

Vigencia: ADR-16, rama de arquitectura. Tres programas ejecutables: Núcleo, Boletería y Trabajador. Operación es el cuarto programa objetivo y sigue sin implementar. Los módulos ausentes no se presentan como servidores terminados. La configuración y las imágenes se validan en infraestructura aislada; el entorno habitual sigue pendiente de corte. Estado y ejecuciones reales en `ESTADO-MIGRACION.md`, apartados 23–24.

Dos portales Angular y una app Flutter unificada para Cliente/Personal consumen el gateway. Sus pantallas o datos simulados no demuestran backend. Gateway verifica JWT con pública y consume el turno; los módulos autorizan y comprueban revocación. Identidad conserva firma/renovación. No existen servidores separados de Pagos, Autenticación, Alimentos, Personal ni los cinco componentes de Boletería.

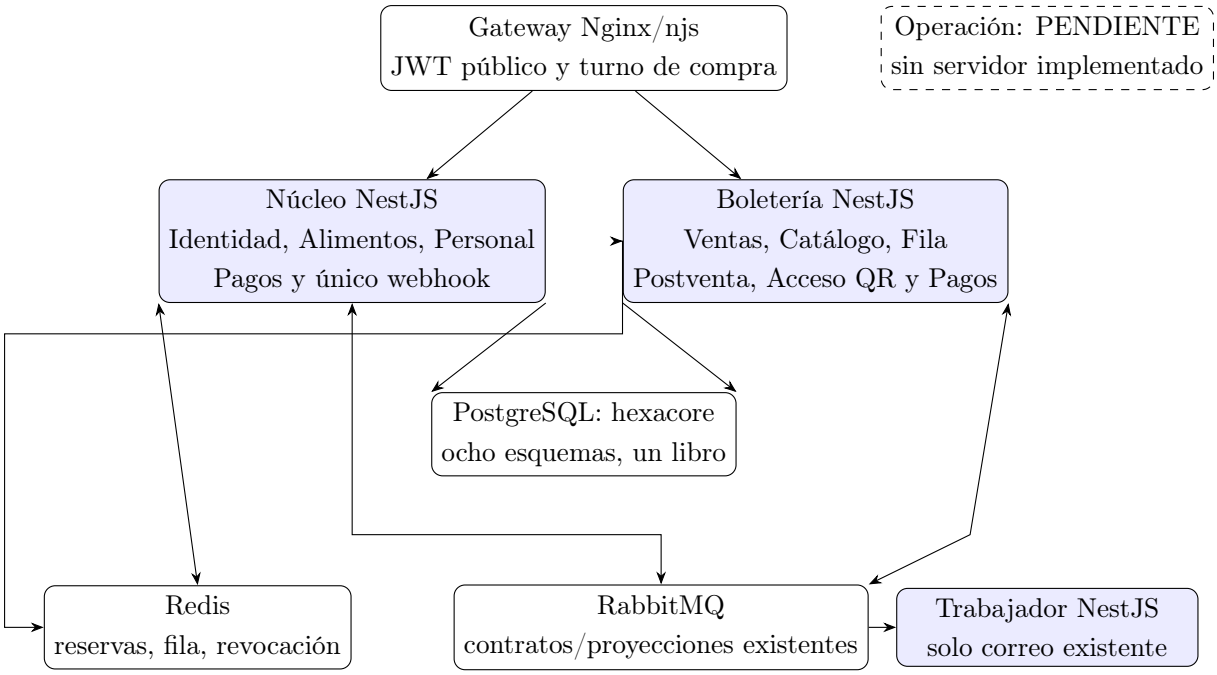


Figura 3: Contenedores vigentes. Azul: programas existentes; discontinuo: objetivo pendiente.

10. Vista de componentes

Cada programa tiene bootstrap real en `App/backend/apps`. Las reglas tienen una sola fuente en `modulos`; dependencias y lockfile comunes, Node 22.22.3, Nest 11/TypeScript 5.9. Los compiladores Angular están fuera del workspace.

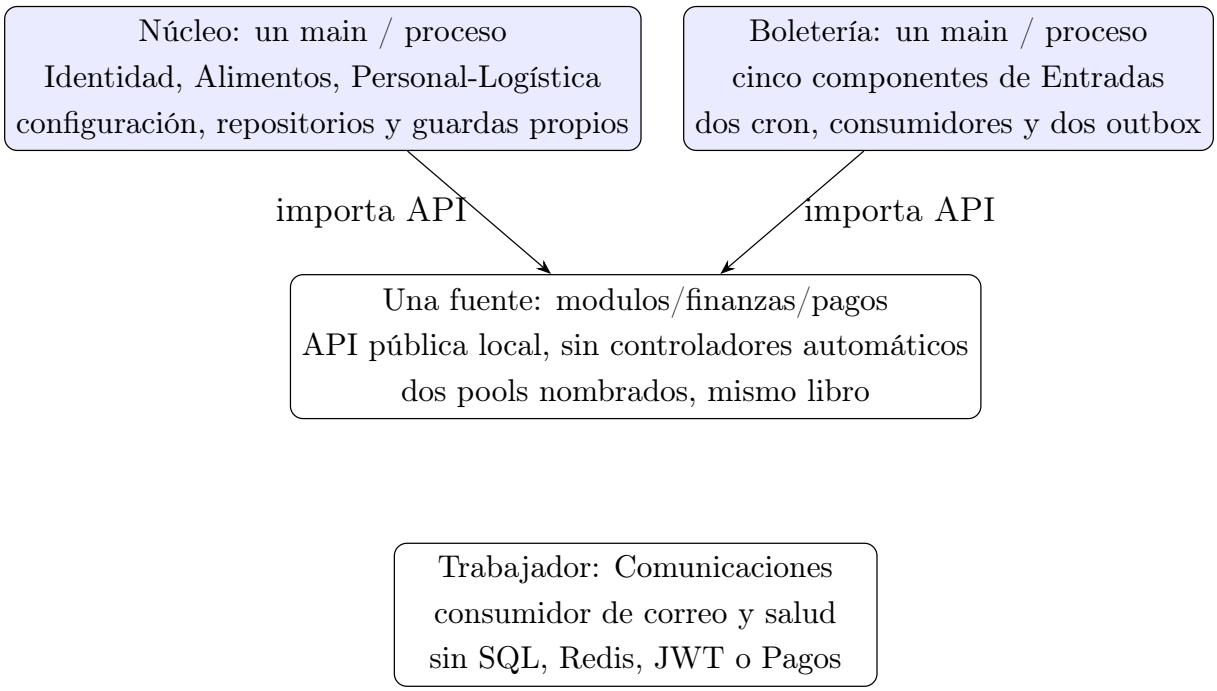
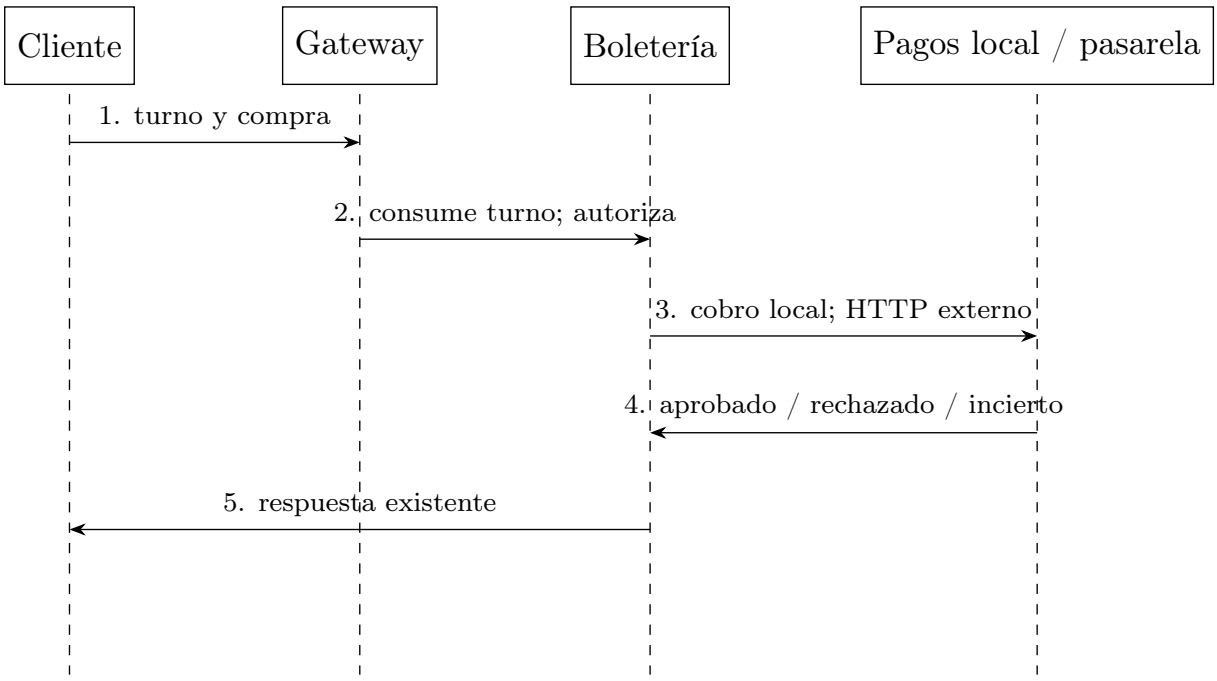


Figura 4: Composición actual. Importar Pagos no publica cobros o reembolsos HTTP.

Núcleo reúne cuentas, sesiones, administración, pedidos y logística; Boletería reúne cartelera, fila, compras, reventa e ingresos. Se conservan rutas `/api/v1`, DTO, estados, errores, cookies, RS256 y reglas previas. El único webhook financiero es `POST /api/v1/pagos/webhook` en Núcleo, con rawBody/HMAC de bytes originales. Boletería no necesita privada JWT ni secreto de webhook. Trabajador consume correo, sin SQL/Redis/JWT/Finanzas y sin endpoints de negocio.

11. Vista de procesos



Libro compartido; QR/ingreso convergen por eventos.
Webhook posterior en Núcleo no finaliza negocio pendiente (F12).

Figura 5: Compra existente; no representa una Saga nueva ni finalización F12.

Alimentos y Entradas llaman a la API pública local de Pagos; solo la pasarela mantiene HTTP externo. Postventa usa la API local de Ventas, conservando guardas y efectos. Compartir proceso/base no reúne automáticamente sus transacciones. Ventas conserva autoridad de propietario/QR/historial; Acceso QR mantiene proyección y registro de ingreso. Continúan eventos, compensaciones y dos outbox. La invalidación inmediata del QR no se acredita con una espera de convergencia.

Contrato existente	Dueño / proceso	Alcance comprobado
Correo pendiente	Identidad / Núcleo; Comunicaciones / Trabajador	Cinco correos por RabbitMQ; intentos/deduplicación en memoria.

Contrato existente	Dueño / proceso	Alcance comprobado
Compra, entrada e ingreso	Ventas y Acceso QR / Boletería	Proyecciones y outbox conservados; no consistencia inmediata nueva.
Cambio/asignación de turno	Personal-Logística / Núcleo	Notificación persistida; listado global F14.
Resultado financiero	Webhook / Núcleo	Libro y publicación; finalizar pendientes sigue F12.
Pedido confirmado	Alimentos / Núcleo	Publicación existente; recepción final pendiente.
Evacuación y alertas	Operación, pendiente	No hay consumidor o caso terminado que migrar.

Tabla 8: Mensajería vigente; no todo publicador tiene outbox.

Cron de reservas/publicaciones, sondeo de alimentos y consumidores de logística/ Entradas permanecen con sus reglas/repositorios en Núcleo/Boletería. Solo el correo se traslada al Trabajador. SIGTERM cierra pools, cron, consumidores y conexiones; el correo en vuelo conserva su posible reentrega (F15).

12. Vista física

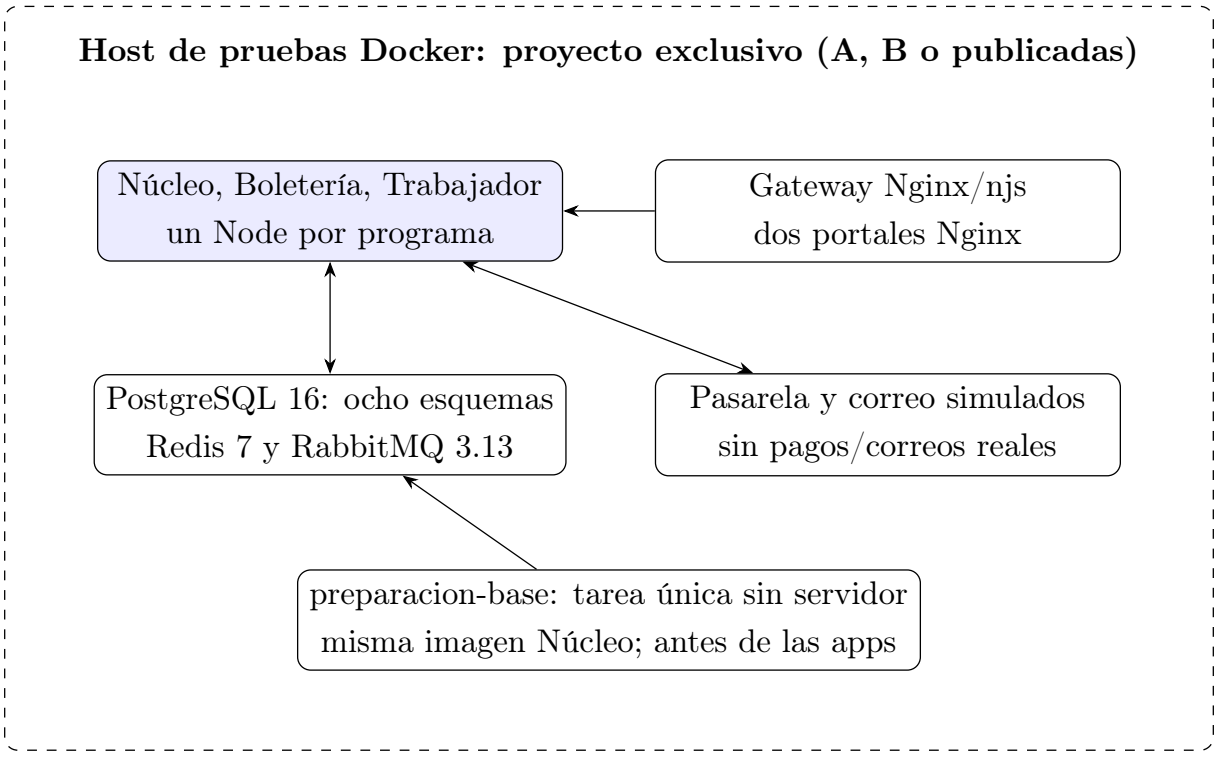


Figura 6: Despliegue comprobado en pruebas; no representa el entorno habitual migrado.

Docker Compose ejecuta los tres programas, gateway, PostgreSQL, Redis, RabbitMQ, dos proveedores simulados y dos portales. `preparacion-base` es una tarea sin listener: ejecuta las migraciones antes de habilitar ambas apps. No es un cuarto servidor. El override remoto separa datos y procesos; escalado replica Boletería; registro reutiliza la misma imagen Núcleo en preparación. Configuración válida no demuestra arranque en dos máquinas ni capacidad, redundancia o RNF-15. No hay Kubernetes, MongoDB, buscador ni réplica SQL desplegados por esta migración. Los diagramas de clúster anteriores son históricos, no evidencia de implementación.

13. Modelo de datos vigente

Un destino PostgreSQL común `POSTGRES_BD=hexacore`; las conexiones TypeORM siguen nombradas. Cada pool tiene solo su esquema, pg_catalog y extensiones en search_path. No se mezclan propietarios por semejanza de tablas.

Conexión	Esquema	Historial / migraciones
identidad	identidad	migraciones / 3
alimentos	alimentos	migraciones / 2
personal-logistica	personal_logistica	migraciones / 2
entradas-ventas	entradas_ventas	migraciones / 5
entradas-catalogo	entradas_catalogo	migraciones / 1
entradas-postventa	entradas_postventa	migrations / 1
entradas-acceso-qr	entradas_acceso_qr	migraciones / 3
finanzas-pagos	finanzas_pagos	migraciones / 1

Tabla 9: Ocho esquemas e historiales; 18 fuentes originales conservadas.

Ambos programas usan `finanzas_pagos.transacciones_pago`, sin claves por proceso ni duplicación del libro. Extensiones compartidas en `extensiones`; procedencia/SHA en `migracion_control`. synchronize/migrationsRun=false. Una preparación serializada reconoce la transferencia sin reaplicar DDL o instala las 18 migraciones en vacío. El mapa tabla por tabla está en la guía de consolidación. La copia transferida y la base vacía se prueban por separado; no se escribe sobre las bases habituales. Respaldos independientes son coherentes entre dominios solo tras inmovilizar conjuntamente escritores/callbacks/consumidores, no por ser pg_dump.

14. Registros de Decisiones Arquitectónicas (ADR)

ADR-01 — Arquitectura de microservicios por dominio

Estado: *superado por ADR-12* — el criterio de separar por dominio sigue vigente; la granularidad de un microservicio por dominio se revisó en ADR-12 a la vista de lo medido sobre el

prototipo desplegado, y el backend queda en tres servicios de dominio en lugar de seis.
ASR relacionados: ASR-06, ASR-09.
Problema: el sistema debe permitir que sus dominios (Entradas, Mercado Secundario, Personal, Eventos, Parqueaderos, Emergencias) evolucionen y escalen de forma relativamente independiente, sin que un cambio en uno arriesgue la disponibilidad de los demás.
Solución: se adopta una arquitectura de **microservicios**, uno por dominio, cada uno con base de datos propia.
Alternativa descartada: monolito modular — dificulta escalar de forma independiente el servicio de Entradas durante una apertura de venta sin sobre-aprovisionar los demás dominios.
Consecuencias: favorece mantenibilidad y escalabilidad independiente, a costa de mayor complejidad operativa (más piezas de infraestructura) y de tener que gestionar consistencia eventual entre servicios en vez de transacciones ACID únicas.

ADR-02 — API Gateway y balanceador de carga

ASR relacionados: ASR-02, ASR-03.
Problema: se necesita un punto único, seguro y disponible de entrada al backend para las tres interfaces del sistema.
Solución: un **API Gateway** centraliza autenticación, autorización y enrutamiento; un **balanceador de carga** distribuye solicitudes entre múltiples instancias de cada microservicio.
Alternativa descartada: exponer cada microservicio directamente a los clientes — se descartó por duplicar lógica de seguridad en cada interfaz y complicar el enrutamiento.
Consecuencias: centraliza seguridad y facilita el enrutamiento, pero introduce un componente crítico que debe replicarse (mitigado con múltiples instancias detrás del balanceador) para no convertirse en un punto único de falla.

ADR-03 — Redis para bloqueo temporal y datos de acceso rápido

ASR relacionados: ASR-01, ASR-05.
Problema: evitar que dos compradores completen simultáneamente la compra de la misma entrada en reventa, y disponer de datos de aforo/ocupación con baja latencia.
Solución: Redis se usa como bloqueo distribuido temporal durante el checkout de una reventa, y como caché de datos de acceso rápido (aforo, ocupación, sesiones).
Alternativa descartada: bloqueo pesimista directamente en la base de datos relacional — se descartó por convertirse en cuello de botella bajo la carga concurrente esperada.
Consecuencias: resuelve la doble venta y acelera consultas frecuentes, pero añade un punto adicional de falla que requiere su propia estrategia de disponibilidad (réplica/clúster).

ADR-04 — Cola de mensajes para operaciones asíncronas

Estado: *completado por ADR-10* — la decisión de usar una cola sigue vigente; el motor concreto, que aquí quedó abierto como "RabbitMQ/Kafka", se cerró en ADR-10 con evidencia medida.
ASR relacionados: ASR-04, ASR-07.
Problema: operaciones como notificaciones o liquidación de pagos no deben bloquear ni hacer

fallar la operación principal (compra, transferencia) cuando un servicio externo es lento o falla temporalmente.
Solución: una **cola de mensajes (RabbitMQ/Kafka)** desacopla la generación de QR, las notificaciones, la auditoría y la liquidación de pagos del flujo síncrono principal.
Alternativa descartada: comunicación síncrona directa con los servicios externos — se descartó porque un fallo temporal del proveedor de notificaciones no debe bloquear la operación principal.
Consecuencias: tolera fallos externos y mejora el rendimiento percibido, a costa de consistencia eventual y de requerir monitoreo propio de las colas.

ADR-05 — Stack de frontend por tipo de interfaz

Estado: *parcialmente superado por ADR-08* — lo relativo a los portales web sigue vigente; la decisión sobre el stack de la app móvil fue revertida el 2026-09-07. Se conserva el texto original como registro de por qué se decidió así en su momento.
ASR relacionados: ASR-08, ASR-10, ASR-05.
Problema: las tres interfaces (portal web del cliente, portal web administrativo/operativo, app móvil) deben implementarse en un lenguaje/framework concreto, sin comprometer el consumo uniforme del backend vía API Gateway (ASR-10) ni la especialización por rol de cada interfaz (ASR-08).
Solución: los portales web se construyen en **Angular + TypeScript** — su estructura de módulos, *guards* e *interceptors* favorece el trabajo en paralelo de varios integrantes con convenciones consistentes, y RxJS encaja con las actualizaciones en tiempo real de ASR-05. La app móvil se construye en **Kotlin nativo sobre Android Studio** (Jetpack Compose para la interfaz, *coroutines/Flow* para tiempo real), aprovechando que el equipo ya domina ambos lenguajes; sirve a los roles Cliente y Personal desde un mismo proyecto (ver ADR-07), por lo que ASR-08 se resuelve con navegación condicionada por rol en vez de con binarios separados.
Alternativa descartada: un framework híbrido/*cross-platform* (p. ej. React Native o Flutter) para compartir código de interfaz entre web y móvil — se descartó porque ASR-10 ya evita duplicar lógica de negocio en el cliente (esta vive en el backend, detrás del API Gateway), por lo que compartir código de UI entre plataformas no aporta un beneficio arquitectónico claro frente al costo de una capa de abstracción adicional y a la experiencia ya existente del equipo con Angular y Kotlin nativos.
Consecuencias: cada interfaz evoluciona con las herramientas nativas de su plataforma sin las limitaciones de un framework híbrido, a costa de mantener dos ecosistemas de *build*/ pruebas distintos (TypeScript/npm y Kotlin/Gradle) y de no compartir código de UI entre web y móvil — los contratos comunes quedan limitados a la especificación de la API (ver `App/shared/`).

ADR-06 — Motor de base de datos por dominio (*polyglot persistence*)

ASR relacionados: ASR-01, ASR-07.
Problema: ADR-01 estableció una base de datos propia por microservicio, pero dejó abierto el motor concreto; los esquemas transaccionales (Entradas, Personal, Eventos/Emergencias, Parqueaderos, cuentas/proveedores/pagos) tienen necesidades de consistencia distintas a las de Reportes/analítica, cuyos datos son más variables y semi-estructurados.

Solución: PostgreSQL como motor relacional para todos los esquemas transaccionales (uno por microservicio, ADR-01), y **MongoDB** como motor documental exclusivamente para el esquema de Reportes dentro del servicio de administración.
Alternativa descartada: un único motor relacional (PostgreSQL) también para Reportes — se descartó para no forzar un esquema rígido sobre datos de reportes cuya estructura varía entre tipos de reporte, tal como ya anticipaba la sección de modelo de datos.
Consecuencias: cada esquema usa el motor más adecuado a su naturaleza (ACID para lo transaccional, esquema flexible para reportes), a costa de operar y respaldar dos motores de base de datos distintos en vez de uno solo.

ADR-07 — App móvil unificada para Cliente y Personal

ASR relacionados: ASR-08, ASR-10.
Problema: el diseño original (v1.1) proponía dos apps móviles nativas independientes, una por rol (Cliente y Personal). Al construir el prototipo de la Entrega 2 con un equipo pequeño, mantener dos proyectos Android desde cero duplica configuración de Gradle, tema visual y esqueleto de navegación sin que exista todavía una razón operativa concreta (distribución, permisos de dispositivo, política de actualización) que obligue a separarlos en esta etapa.
Solución: una única app (`App/frontend/app-movil`) con una pantalla de login compartida. Las credenciales identifican el rol del usuario autenticado y la navegación interna abre el conjunto de pantallas del rol correspondiente: Inicio/Entradas/Parqueadero/Pedidos para Cliente, Turnos/Asistencia/Validación para Personal. Esta decisión sigue vigente tras el cambio de stack de ADR-08: la app Flutter mantiene el mismo esquema de una sola app para ambos roles.
Alternativa descartada: mantener **App Móvil del Cliente** y **App Móvil del Personal** como dos módulos/APKs independientes (diseño de la v1.1) — se descartó para este prototipo porque el beneficio principal de separarlas (aislar del binario público de clientes las pantallas y permisos operativos del personal, y poder distribuirlas y versionarlas por canales distintos — tiendas públicas vs. MDM interno) todavía no tiene un requisito concreto que lo exija, y el costo de mantener dos proyectos no se justifica en esta fase.
Consecuencias: menos código y configuración duplicados mientras el equipo es pequeño, y un único punto de login para ambos roles. A cambio, el binario que instala un cliente cualquiera desde la tienda pública contiene también las pantallas y — potencialmente — los permisos operativos del personal (aunque inaccesibles sin credenciales válidas), lo que amplía la superficie de la app frente a la alternativa separada; y ambos roles quedan acoplados a un mismo ciclo de release. Si el rol Personal termina necesitando permisos de dispositivo, distribución o cadencia de actualización distintos a los del Cliente, esta decisión debe revisarse y volver a separar los binarios.

ADR-08 — Migración de la app móvil a Flutter

ASR relacionados: ASR-08, ASR-09, ASR-10.
Problema: ADR-05 fijó **Kotlin nativo** para la app móvil y descartó explícitamente los frameworks *cross-platform*, con el argumento de que compartir código de interfaz no aportaba un beneficio arquitectónico claro. Ese razonamiento omitió un requisito que sí es del proyecto: la

app debe llegar a **Android y iOS**. Con el stack nativo, cubrir iOS obliga a escribir y mantener una segunda implementación completa (Swift/SwiftUI) de las mismas pantallas, lo que para un equipo de cuatro personas duplica el esfuerzo de toda evolución de la interfaz y multiplica el riesgo de que ambas plataformas se desincronicen.
Solución: la app móvil se reimplementa en **Flutter** (Dart), reemplazando la versión Kotlin/Jetpack Compose. Una sola base de código produce los binarios de Android e iOS, conservando el esquema de ADR-07 (una sola app para los roles Cliente y Personal, con el rol resuelto en el login). La implementación incluye un cliente de API centralizado (`lib/services/api_client.dart`) que concentra el consumo del backend, de modo que la futura integración con el API Gateway quede en un solo punto y no disperso por las pantallas (ASR-10).
Alternativas consideradas:

- **Mantener Kotlin y añadir una app iOS nativa en Swift/SwiftUI** — máxima fidelidad al look-and-feel de cada plataforma y acceso inmediato a APIs nuevas del sistema operativo, pero duplica la base de código de interfaz y el esfuerzo de mantenimiento (Modificabilidad, ASR-09).
- **Kotlin Multiplatform (KMM)** — comparte la lógica de negocio y de red pero mantiene la interfaz nativa por plataforma; es el término medio, y se descartó porque el grueso del trabajo de este prototipo está justamente en la interfaz, que KMM no comparte.
- **Flutter** (elegida) — una sola base de código para interfaz y lógica en ambas plataformas.

Consecuencias: una sola base de código para Android e iOS, y un único ciclo de build y pruebas para la app móvil, a costa de: (1) renunciar a los widgets nativos reales — Flutter dibuja su propia interfaz con su motor de render, de modo que la fidelidad con cada plataforma es una aproximación, no el control nativo; (2) sumar un tercer ecosistema de herramientas (Dart/pub) al del web (TypeScript/npm) y el del backend; y (3) una curva de aprendizaje en Dart para el equipo. La implementación Kotlin previa se elimina del repositorio; queda en el historial de Git si hiciera falta recuperarla.
Nota sobre la evidencia: esta decisión se tomó reimplementando la app completa en Flutter y comparándola con la versión Kotlin ya existente, no solo con un análisis en papel. La comparación sistemática de métricas entre ambas implementaciones (tiempo de desarrollo, líneas de código, arranque en frío, fidelidad visual) queda pendiente de documentarse como evidencia formal para la Entrega 1.

ADR-09 — Lenguaje y framework del backend

ASR relacionados: ASR-04, ASR-06, ASR-09, ASR-14.
Problema: `App/services/*` estaba marcado como "lenguaje por definir". Es la única decisión que bloquea empezar a codificar los seis microservicios, y debía tomarse con evidencia medida — no por preferencia ni por popularidad.
Solución: los microservicios de dominio se implementan en **NestJS** (Node.js + TypeScript).
Alternativas consideradas: NestJS y **Spring Boot** (Java 17) se compararon construyendo el mismo endpoint en ambos y midiéndolos bajo carga (PoC-04, ver `App/PoCs/poc-04-lenguaje-backend/`).

Se descartaron sin PoC: **Go**, porque nadie en el equipo lo conoce y sumar un lenguaje nuevo con la entrega encima es el mayor riesgo de cronograma; y **Python/FastAPI**, por su tipado opcional, poco conveniente con seis servicios desarrollados en paralelo por cuatro personas.
Evidencia (PoC-04): mismo endpoint, misma respuesta byte por byte, 100 conexiones concurrentes, 15 s por medición, cero errores:

Candidato	Escenario	req/s	p50 (ms)	p97,5 (ms)	RSS (MB)
NestJS	sin I/O	21.071	4	5	156
NestJS	con 2 ms de I/O	18.530	5	6	158
Spring Boot	sin I/O	91.441	0	1	335
Spring Boot	con 2 ms de I/O	32.288	3	4	337

Arranque hasta la primera respuesta: **NestJS 0,28 s** frente a **Spring Boot 1,09 s**.

Tres lecturas de esos números:

1. **Ambos cumplen RNF-07 con un margen enorme.** El umbral es p95 $\leq$ 500 ms y el peor caso medido fue 7 ms: setenta veces por debajo. El rendimiento, por lo tanto, **no es el criterio que decide** esta elección — ninguno de los dos está cerca de incumplirlo.
2. **Spring Boot rinde más, y conviene decirlo sin matizarlo:** 4,3× sin I/O. Pero la ventaja cae a 1,7× al añadir 2 ms de I/O simulada, y contra una base de datos real (5–20 ms) se encogería más. Cuanto más se parece el escenario a producción, menos pesa la diferencia.
3. **NestJS arranca 4× más rápido y usa menos de la mitad de memoria**, lo que favorece a RNF-15 (despliegue sin interrupción) y al escalado elástico de ASR-06.

Por qué se elige NestJS pese a que Spring Boot rinde más: porque el rendimiento ya está resuelto por ambos con setenta veces de margen, y los criterios que sí quedan abiertos favorecen a NestJS. En **Modificabilidad** (ASR-09) y **Comprobabilidad** (ASR-14, RNF-18), NestJS impone módulos e inyección de dependencias — el mismo modelo que el equipo ya aplica en los portales Angular, y el que hace que sustituir dependencias externas por dobles de prueba sea directo. Y el riesgo real del proyecto no es la latencia sino el cronograma: cuatro personas deben entregar seis microservicios, y usar el lenguaje que ya escriben en el frontend elimina un cambio de contexto completo.
Consecuencias: un solo lenguaje entre los portales web y el backend, con contratos de API que pueden generar tipos para ambos lados (`App/shared/`). A cambio se acepta un techo de rendimiento más bajo que el de Spring Boot — irrelevante frente a los umbrales actuales, pero que debería reevaluarse si algún servicio pasa a ser intensivo en CPU (los reportes con Map-Reduce, hoy delegados a MongoDB, son el candidato a vigilar). Node ejecuta en un solo hilo por proceso, de modo que el escalado es horizontal por réplicas, que es justamente lo que establecen ADR-01 y ASR-06.
Validez de la evidencia: el PoC mide un endpoint sin base de datos real, con cliente y servidor en la misma máquina, y no llegó a los 2.000 usuarios concurrentes de RNF-07 (con 100 conexiones el generador de carga ya era el cuello de botella). Tampoco mide lo que probablemente más pesa

en la práctica: la velocidad de desarrollo del equipo. **Si alguno de los integrantes tuviera experiencia sólida en Java**, esta decisión sería genuinamente discutible y Spring Boot quedaría bien defendido por los datos.

ADR-10 — Motor de la cola de mensajes: RabbitMQ

ASR relacionados: ASR-04, ASR-07, ASR-13.
Problema: ADR-04 estableció que hace falta una cola de mensajes, pero dejó el motor sin decidir, escrito como "RabbitMQ/Kafka" — como si fueran intercambiables. No lo son: responden a modelos distintos (cola de tareas frente a registro de eventos) y la elección condiciona cómo se escriben los consumidores en los seis microservicios.
Solución: RabbitMQ.
Evidencia (PoC-05, `App/PoCs/poc-05-cola-mensajes/`): se publicó y consumió el evento real `ENTRADA_TRANSFERIDA` en ambos brokers (los dos en Docker, 500 mensajes por combinación), midiendo la latencia de extremo a extremo:

Broker	Modo	p50 (ms)	p95 (ms)	msg/s
RabbitMQ	rápido (sin persistir)	**16,3**	**17,5**	18.932
Kafka	rápido (`acks=0`)	32,1	36,0	12.418
RabbitMQ	duradero (confirmado)	162,0	282,1	1.619
Kafka	duradero (`acks=all`)	**31,6**	**57,9**	7.702

El ganador se invierte según el modo, y conviene decirlo sin matizarlo: sin durabilidad RabbitMQ es el doble de rápido, pero **con durabilidad Kafka es cinco veces mejor** (31,6 ms frente a 162 ms de mediana). La razón es estructural: RabbitMQ hace `fsync` por mensaje al persistir y su latencia se multiplica por diez, mientras que Kafka escribe a un log secuencial y la durabilidad le sale prácticamente gratis. Y el modo que importa aquí es el duradero: ni una transferencia de entrada ni una alerta de evacuación pueden perderse porque el broker se reinició.

Sin embargo, **ambos cumplen RNF-17 con enorme margen**: el umbral es 10 s y el peor caso medido —RabbitMQ duradero, p95— fue **282 ms**, treinta y cinco veces por debajo. Igual que en ADR-09, la latencia no es el criterio que decide.

Se midió entonces lo que sí queda abierto, con el mismo escenario en ambos (10 eventos, 3 rechazados a propósito):

	RabbitMQ	Kafka
Cola de mensajes muertos (DLQ)	nativa, 5 líneas declarativas	no existe: hay que construirla
Imagen / memoria en reposo	275 MB / 147 MB	634 MB / 309 MB
Líneas para levantarlo	9	15
Consola de administración	incluida	no incluida
Conceptos a dominar	colas, *exchanges*, *bindings*	+ particiones, offsets, *consumer groups*, ret

Por qué RabbitMQ pese a que Kafka es más rápido en modo duradero: porque el uso

que este sistema le da a la cola es el de una **cola de tareas** —disparar una notificación, generar un QR, registrar una auditoría, avisar de una evacuación— y no el de un *registro de eventos* con reproducción histórica y múltiples consumidores independientes, que es donde Kafka se justifica. Para ese uso, RabbitMQ ofrece de fábrica lo que aquí sí hace falta: **DLQ y reintentos nativos**, que ASR-07 y ASR-13 exigen para no perder en silencio un evento que falló. En Kafka eso hay que construirlo y mantenerlo a mano. A eso se suma la mitad de memoria, la mitad de configuración y una consola incluida, con un equipo de cuatro personas que además debe operar PostgreSQL, MongoDB y Redis.

Consecuencias: se acepta una latencia mayor en modo duradero (282 ms de p95 frente a los 58 ms de Kafka) — irrelevante frente al umbral de 10 s de RNF-17 — y un techo de throughput más bajo (1.619 msg/s duraderos). Ese techo es holgado para el volumen del sistema: los eventos se generan por transferencia, pedido o alerta, no de forma continua. **Cuándo habría que revisar esta decisión:** si aparece la necesidad de reproducir el histórico de eventos, de que varios consumidores independientes lean el mismo flujo, o si el volumen se acerca al millar de mensajes por segundo sostenido. En cualquiera de esos casos Kafka pasa a estar mejor justificado, y los números de este PoC lo respaldarían.

Validez de la evidencia: un solo nodo de cada broker (en producción irían replicados, y ahí Kafka replica mejor), todo en la misma máquina sin red de por medio, y 500 mensajes por medición — no se probó el comportamiento con colas de millones de mensajes, que es donde Kafka se separa de verdad. Tampoco se midió la recuperación ante caída del broker, que es justamente el escenario que motiva el modo duradero.

ADR-11 — Autenticación entre servicios: RS256 y dos tokens

Contexto: RNF-06 exige que *toda* petición a un microservicio llegue autenticada y autorizada por rol. Con seis dominios independientes (ADR-01), la pregunta es quién valida la sesión y con qué. El CU-027 añade dos exigencias propias: que cerrar sesión surta efecto de verdad — un JWT es válido hasta que expira, lo firme quien lo firme — y que la sesión no se caiga sola mientras la persona está usando la aplicación.

Alternativas consideradas:

- **HS256 con un secreto compartido.** Fue la primera implementación. Funciona mientras un solo servicio verifica, pero con HS256 firmar y verificar usan la misma clave: cada servicio que valida tokens podría **fabricar uno de administrador**. Con seis microservicios, seis lugares por donde se filtra la llave maestra.
- **Consultar al servicio de Administración en cada petición.** Autorización siempre exacta, pero convierte a ese servicio en un punto único de fallo para todo el sistema y le suma una llamada de red a cada operación.
- **RS256 con clave asimétrica** — la elegida.

Decisión: tokens JWT firmados con **RS256**. La clave **privada** la tiene solo el servicio de Administración, que es el único que emite; los demás reciben la **pública**, que sirve para verificar

y no para firmar. Al arrancar se comprueba que las dos claves se correspondan, que sean RSA de 2048 bits o más y que en producción no sean las de desarrollo del repositorio.

Dos decisiones acompañan a la principal:

- **Revocación por Redis (ADR-03).** Cada sesión cerrada se publica como `sesion-revocada:<jti>` con una vida igual a lo que le quedaba al token. Así los demás servicios rechazan un token revocado sin preguntar a Administración, y la lista no crece: cuando la clave expira, el token ya no valía por sí solo.
- **Dos tokens.** Uno de **acceso** de 15 minutos, que viaja en cada petición, y uno de **renovación** de 30 días sin uso, que solo viaja al renovar. Cada renovación emite uno nuevo e invalida el anterior; reutilizar uno gastado cierra la sesión entera, porque significa que dos partes tienen copia. En el navegador el token de renovación va en una cookie `HttpOnly; SameSite=Strict`, fuera del alcance de cualquier script inyectado.

Atributo de calidad: Seguridad (ASR-03) como motivación principal; Disponibilidad (ASR-02), porque validar sin consultar evita el punto único de fallo; y Usabilidad (ASR-08), porque los dos tokens evitan que la sesión se cierre sola con la aplicación abierta.

Evidencia: implementado y probado sobre el sistema real. Las pruebas de `App/services/*/pruebas/` comprueban que un token con `alg: none` o firmado con la clave pública como secreto HMAC se rechaza; que una sesión cerrada deja de valer *en el otro microservicio*; y que reutilizar un token de renovación ya gastado cierra la sesión. El API Gateway (ADR-02) valida la firma con la clave pública sin consultar a Autenticación (ADR-15), y una prueba de defensa en profundidad confirma que llamar directo a un servicio con la cabecera de identidad falsificada devuelve 401.

Consecuencias: firmar con RSA cuesta más CPU que con HMAC. A cambio, ningún servicio distinto del emisor puede fabricar un token. Hasta ADR-15 el gateway validaba con una subpetición a `/sesiones/verificar` en cada petición protegida; esa era la condición de revisión de este ADR, y se ejerció: ahora valida la firma por su cuenta, y la revocación la siguen comprobando en Redis los servicios.

ADR-12 — Arquitectura basada en servicios: un servicio de dominio agregado y dos replicables

Estado: *superado por ADR-14* — el estilo basado en servicios sigue vigente; lo que se revisó es el criterio de extracción, que aquí tenía dos condiciones y en ADR-14 pasa a cuatro. Con el criterio nuevo, Autenticación sale del Núcleo como microservicio y Alimentos queda como servicio de dominio. **Registro histórico**: describía el despliegue al decidirse ADR-12; la rama actual sigue ADR-16 y el corte habitual está pendiente.
Contexto: ADR-01 fijó seis microservicios, uno por dominio, cada uno con base de datos propia. Al construir el prototipo funcional aparecieron tres hechos medibles sobre el sistema desplegado que esa decisión no previó:

- **Solo uno de los seis se replica.** El fichero `App/infra/docker-compose.escalado.yml` existe únicamente para quitarle el puerto fijo al servicio de Entradas y poder correr `--scale=2`.

Los otros cinco declaran `container_name` y puerto publicados, que por construcción impiden una segunda instancia: la independencia de despliegue de seis servicios compró escalado independiente de uno.

- **Dos de los seis no existen.** *Personal* y *Parqueaderos* no pasaron de su `README`. La granularidad elegida excedió la capacidad real del equipo, y con ella la cobertura de pruebas de esos dos dominios es del **0%**.
- **La separación de datos ya es lógica, no física.** Las seis bases se crean sobre una sola instancia de PostgreSQL (`App/infra/initdb/01-bases.sql`). El sistema paga el coste de coordinar entre dominios — *Pago* en una base y *Pedido* en otra, reconciliados a nivel de aplicación — sin obtener el aislamiento de fallos que justificaría pagarlo.

El diagnóstico no es que ADR-01 eligiera mal el criterio, sino que lo aplicó a una granularidad que los requisitos no piden: en el vocabulario del *Scale Cube*, se descompuso por función (eje Y) los seis dominios cuando solo uno tiene una curva de carga distinta del resto.

Alternativas consideradas:

- **Mantener ADR-01 y corregir solo la replicabilidad.** Quitar `container_name` y los puertos fijos a los servicios que no los necesitan. Cuesta una tarde y cierra el defecto operativo, pero deja cinco unidades desplegables cuya curva de carga no justifica el coste de mantenerlas, y no responde a la objeción de fondo.
- **Monolito modular** — una sola unidad desplegable con fronteras internas. Rompe RNF-10: no permite duplicar el servicio de Entradas durante una apertura de venta sin duplicar también Pedidos, Personal y Parqueaderos.
- **Monolito replicado en bloque** (eje X puro). Absorbe la carga, pero sobre-aprovisiona: cada réplica arrastra los módulos que nadie está usando.
- **Arquitectura basada en servicios** — la elegida.

Decisión: se adopta una **arquitectura basada en servicios** (*Service-Based Architecture*), con partición por dominio, granularidad gruesa y la variante de **base de datos particionada por dominio**. El backend queda en **tres servicios de dominio** en lugar de seis:

- **Núcleo** — una unidad desplegable que contiene *Administración* y *Pedidos* como módulos, y donde se incorporarán *Personal* y *Parqueaderos* cuando se implementen. Cada módulo conserva su esquema de datos, su `data-source` y su propia secuencia de migraciones, de modo que un cambio de esquema en un dominio sigue sin tocar a los demás.
- **Entradas / Mercado Secundario** — servicio independiente y replicable.
- **Eventos / Emergencias** — servicio independiente y replicable.

El **API Gateway** (ADR-02), **Redis** (ADR-03), la **cola de mensajes** (ADR-10) y el esquema de **dos tokens RS256** (ADR-11) no cambian, y tampoco cambia ninguna ruta pública: el

gateway pasa de enrutar hacia dos destinos a enrutar hacia uno, con lo que los portales web y la app móvil no se modifican. Esta decisión **supera a ADR-01**.

Criterio de extracción. Un dominio permanece como servicio independiente —y no como módulo del núcleo— solo si cumple al menos una de estas dos condiciones:

1. **Su curva de carga difiere en orden de magnitud de la del resto**, por lo que necesita réplicas propias. Es el caso de Entradas: RNF-07 exige p95 $\leq$ 500 ms con 2.000 usuarios concurrentes en la apertura de venta, y RNF-10 exige que al duplicar sus instancias el *throughput* crezca $\geq$ 1,7×.
2. **Su disponibilidad no puede depender de la del resto del sistema.** Es el caso de Eventos / Emergencias: RNF-17 exige que una evacuación llegue al $\geq$ 99% de los dispositivos en $\leq$ 10 s, y RNF-09 que el tablero de aforo refleje un ingreso en $\leq$ 2 s. Si el núcleo deja de responder, la evacuación debe seguir funcionando.

Ninguno de los cuatro dominios restantes cumple ninguna de las dos condiciones: su carga es plana y comparable entre sí, y su caída no compromete la seguridad física de nadie. De ahí que los satélites sean exactamente dos, y no tres ni cuatro. El criterio es además la regla con la que se decidirá dónde ubicar cualquier dominio que entre al sistema más adelante.

Dos decisiones acompañan a la principal:

- **Fronteras de módulo verificadas por herramienta.** Dentro del núcleo, lo que antes garantizaba el límite de proceso lo garantiza ahora una regla de *linting* y una prueba automática que fallan si un módulo importa de otro algo que no sea su API pública. Esto es *más* verificable que la situación anterior, en la que nada impedía que un servicio consultara la base de datos de otro.
- **El núcleo arranca con una sola instancia.** Administración y Pedidos ya corren hoy en instancia única, de modo que un núcleo de una réplica no degrada la disponibilidad respecto al estado actual. La segunda réplica queda como iteración posterior porque exige resolver antes la ejecución de trabajos en segundo plano (ver *Consecuencias*).

Atributo de calidad: Mantenibilidad (ASR-09) y simplicidad operativa como motivación principal — se pasa de seis unidades desplegables a tres sin perder ninguna capacidad medida; **Escalabilidad** (ASR-06), que se preserva intacta porque el único dominio que exigía escalar de forma independiente sigue haciéndolo; **Consistencia** (RNF-01), que mejora porque los módulos del núcleo pueden compartir una transacción en lugar de reconciliarse a nivel de aplicación; y **Comprobabilidad** (RNF-18), porque un módulo se prueba con dobles en proceso sin levantar contenedores.

Evidencia (al decidirse ADR-12): la justificación era verificable sobre el repositorio y sobre el sistema desplegado. Que solo un servicio admitía réplicas se comprobaba en `docker-compose.escalado.yml` y en los `container_name` del `docker-compose.yml`; que dos dominios no estaban implementados, en los módulos `personal/` y `parqueaderos/` del núcleo; que las seis bases comparten instancia, en `initdb/01-bases.sql`; y que **no había una sola llamada HTTP síncrona entre servi-**

cios —lo que distinguía esa arquitectura de un monolito distribuido— en que los servicios solo se comunicaban por el gateway, por la cola de mensajes y verificando el token con la clave pública.

Qué de esto cambió con ADR-14. Dos de las cuatro evidencias ya no describen el sistema, y conviene decirlo aquí en vez de dejar el párrafo en presente:

- **Las rutas de los dominios sin implementar.** El núcleo se repartió y se renombró: esos dos módulos están hoy en `App/services/autenticacion/src/modulos/`, con sendos marcadores en `App/services/personal/` y `App/services/parqueaderos/`. Siguen sin implementarse.
- **Ya hay llamadas HTTP síncronas entre servicios, y son deliberadas.** Al extraer Acceso QR y Postventa, la tabla `entradas` se quedó en Ventas, que es su dueño. Los dos servicios la necesitan y se la piden por una API interna (`/interno/entradas/`) que el gateway no enruta. Postventa llama además a Pagos.

  Esto no convierte la arquitectura en un monolito distribuido, pero la afirmación de arriba ya no es el argumento que lo demuestra. Lo que lo sostiene ahora es **qué** se hace síncrono y qué no: las llamadas síncronas son las que no pueden ser de otra manera porque alguien espera una respuesta —gastar un QR en la puerta, transferir una entrada con el pago hecho— y cada una está envuelta en una Saga con compensación explícita, con un resultado «no respondió» distinto de «dijo que no». Todo lo demás —notificar, liquidar, proyectar— sigue yendo por el broker. La diferencia con un monolito distribuido no es el número de llamadas síncronas: es que ninguna de ellas lee la base de otro servicio, y que cada una tiene definido qué pasa cuando falla.

La línea base contra la que se validará la migración es la de `App/infra/informe-atributos/INFORME.md`: p95 de **189,9 ms** con 200 peticiones simultáneas (umbral 500 ms), recuperación tras `kill -9` en **1,3 s** (umbral 30 s), **0%** de rutas de negocio alcanzables sin token, y **1 de 50** compras concurrentes aceptada sobre la misma publicación. La condición de aceptación de la migración es que `App/infra/atributos-calidad.py` devuelva cifras iguales o mejores que estas. *Esa medición posterior está pendiente: a la fecha de este registro la decisión está tomada y la migración no se ha ejecutado.*

Consecuencias: agrupar cuatro dominios en una unidad desplegable tiene tres costes identificados, y los tres tienen una respuesta explícita.

- **RNF-13 deja de cumplirse en su formulación literal.** Su métrica exige «microservicios que deben modificarse y redesplegarse ante un cambio típico de negocio: 1», y un cambio en las reglas de Pedidos redespliega ahora todo el núcleo. La métrica se reformula a «**módulos de dominio que deben modificarse: 1**; unidades desplegables afectadas: 1», porque el atributo que el requisito perseguía es el **bajo acoplamiento**, no el número de procesos — y eso lo garantizan las fronteras verificadas por herramienta.
- **El radio de impacto crece.** Una excepción no atrapada en un módulo puede detener el proceso que también atiende el inicio de sesión, algo imposible mientras eran procesos

separados. El núcleo es *stateless* y está preparado para correr con varias réplicas detrás del balanceador; mientras corra con una sola, su disponibilidad es la misma que la de hoy.

- **Aparecen dos acoplamientos nuevos dentro del proceso.** El primero es el **pool de hilos**: Administración fija `UV_THREADPOOL_SIZE` porque el derivado de contraseñas es bloqueante, y ese pool queda compartido con las consultas de Pedidos; es precisamente lo que debe vigilarse al volver a medir el desempeño. El segundo son los **trabajos en segundo plano**: la caducidad de reservas de inventario de Pedidos es un temporizador, y con dos réplicas se ejecutaría dos veces sobre las mismas reservas. Antes de activar la segunda réplica debe protegerse con un candado en Redis (`SET NX` con vencimiento, ADR-03), de modo que solo la instancia que lo obtiene ejecute el ciclo. Los consumidores de la cola de mensajes no tienen este problema: RabbitMQ reparte cada mensaje a una sola instancia.

Cuándo revisar: si alguno de los módulos del núcleo desarrolla su propia curva de carga o su propio ciclo de despliegue, se le aplica el criterio de extracción y sale como servicio independiente. La estructura elegida —un módulo por dominio, con esquema, `data-source` y migraciones propios— está pensada precisamente para que esa extracción sea una operación de empaquetado y no una reescritura del dominio.

ADR-14 — Arquitectura híbrida entre SOA y microservicios

Vigencia actual: registro histórico. ADR-16 sustituye su topología y revisa las relaciones de Pagos/datos; los mecanismos conservados se enumeran allí.
Estado: *supera a ADR-12.* La separación por dominio sigue vigente; lo que se revisa es el **criterio** con el que se decide qué dominio merece ser un despliegue propio, y el resultado pasa de tres servicios de dominio a **siete microservicios y siete servicios de dominio**.

Contexto: ADR-12 adoptó una arquitectura basada en servicios con un criterio de extracción de dos condiciones — curva de carga e independencia de disponibilidad — y produjo tres servicios de dominio. Al recorrer los seis ASR contra los 32 casos de uso, ese criterio resulta **insuficiente en dos sentidos**:

- **No reconoce los servicios transversales.** Pagos lo usan la compra (CU-001), la reventa (CU-006), los reembolsos (CU-003), los pedidos de alimentos (CU-011) y los parqueaderos (CU-021). Autenticación lo usan todas las peticiones del sistema. Notificaciones lo usa casi todo. Que uno de ellos falle no degrada un dominio: degrada muchos a la vez. ADR-12 no tenía forma de expresar eso.

- **No reconoce el almacén propio.** El Catálogo (CU-005) necesita un índice de búsqueda para filtrar por categoría, fecha, ciudad o artista; la Cola virtual necesita una estructura de fila en memoria; Acceso QR necesita una caché del estado de cada código para validar sin red. Son servicios cuyas réplicas **no competirían** por la base central, precisamente porque su estado no vive ahí.

Hay además un hecho de carga que ADR-12 sí veía pero resolvía a medias: en la apertura de

venta de un evento muy demandado, la mayor parte del tráfico es de **consulta** del catálogo, no de compra. Con ambas cosas en el mismo servicio, las dos se degradan.

Alternativas consideradas:

- **Todo en microservicios.** Un despliegue por dominio para los 32 casos de uso. Multiplica bases de datos, despliegues y monitoreo sin beneficio para dominios de carga estable —personal, parqueaderos, alimentos—. Es exactamente el error que ADR-12 corrigió, y con un equipo pequeño vuelve a dejar dominios sin implementar.
- **Todo en *service-based*** (lo que fijó ADR-12). Con una sola base central y servicios grandes, el pico de la apertura de venta obliga a escalar el sistema entero, y las compras compiten por la misma base que el resto de los dominios.
- **Híbrida entre SOA y microservicios** — la elegida.

Decisión: la base del sistema es *service-based* —servicios grandes por dominio que comparten una base central con un esquema por servicio— y de ella se **extraen como microservicios**, con base de datos propia y réplicas que escalan solas, los servicios que cumplen al menos **dos de cuatro** criterios:

1. **Picos de carga muy por encima del resto:** necesita sumar réplicas sin escalar todo el sistema.
2. **Lo usan varios dominios:** si se cae, afecta a muchos casos de uso a la vez.
3. **Su estado cabe en un almacén propio** —caché, base pequeña, índice de búsqueda—: sus réplicas no compiten por la base central.
4. **Su falla debe quedar aislada:** un problema en él no puede tumbar la operación del evento.

El resultado son **siete microservicios** —Cola virtual, Catálogo, Ventas, Pagos, Autenticación, Acceso QR y Notificaciones— y **siete servicios de dominio** —Eventos y recintos, Postventa, Alimentos, Parqueaderos, Personal y logística, Operación del evento, y Finanzas y reportes—. Todos se exponen por un **único API Gateway** (ADR-02, que no cambia) y se comunican de forma asíncrona por un **broker de eventos**.

Tres decisiones acompañan a la principal:

- **Los servicios de dominio también corren redundados, pero no escalan.** Dos instancias para disponibilidad, número fijo. Operación del evento —que atiende la evacuación, CU-010— es crítico para la disponibilidad aunque su carga sea baja: necesita redundancia, no réplicas elásticas.
- **Finanzas y reportes lee de una réplica de lectura** de la base central, no de la principal. Es lo que impide que sus consultas pesadas (CU-032) degraden la operación durante el evento, que es el sexto ASR.

- **La compra se resuelve con una Saga orquestada por Ventas**, con reserva atómica y vencimiento en la caché, y con llaves de idempotencia en Pagos. La compra termina completa o se deshace sin dejar entradas bloqueadas; un reintento del comprador o un *webhook* reenviado por la pasarela no cobran dos veces (CU-001B, CU-001C, CU-001D).

Atributo de calidad: Rendimiento y **Escalabilidad** como motivación principal —los cuatro servicios del camino de compra escalan sin arrastrar al resto—; **Disponibilidad**, porque aislar Pagos, Autenticación y Notificaciones acota el radio de cada falla; **Consistencia**, por la Saga y la reserva atómica; y **Modificabilidad**, porque los servicios se enteran de los cambios por eventos y añadir un consumidor nuevo no obliga a tocar a quien los publica.

Consecuencias: es la decisión más cara del documento y conviene no disimularlo.

- **De 3 unidades desplegables a 14.** La complejidad operativa que ADR-12 redujo vuelve a subir, ahora de forma deliberada y acotada: solo siete servicios escalan, y los otros siete comparten base y ciclo de despliegue.
- **El Núcleo de ADR-12 se deshace.** Aquel agrupó Administración y Pedidos en un proceso; aquí Autenticación es microservicio con base propia y Alimentos es servicio de dominio sobre la base central. Es el único punto donde esta decisión *revierte* trabajo ya hecho, y está anotado como tal.
- **Cambia la infraestructura, no solo el código.** El broker pasa a uno con tópicos replicados, la orquestación pasa a un clúster con autoescalado horizontal, el Catálogo requiere un índice de búsqueda y los frontends se sirven por CDN. Nada de eso existe hoy.
- **Riesgo de consistencia eventual.** Catálogo se alimenta por eventos de Eventos y recintos, así que un evento recién publicado tarda en aparecer en la búsqueda. Es aceptable para CU-005 y no lo es para el inventario, que por eso vive en Ventas con reserva atómica.

Estado de implementación. A la fecha de este registro la decisión está **tomada y no ejecutada**. Lo desplegado sigue siendo lo de ADR-12: tres servicios de dominio, RabbitMQ, Docker Compose y seis bases separadas. La distancia entre este diseño y el código está medida en `Documentation/Work/MIGRACION-ADR-14.md`, que ordena el camino por fases y dice qué depende de qué.

Cuándo revisar: si al medir la apertura de venta con carga real alguno de los cuatro servicios del camino de compra no necesitara escalar de forma independiente, vuelve a ser candidato a servicio de dominio. El criterio de los cuatro puntos se aplica igual a cualquier dominio que entre al sistema después.

ADR-15 — Correcciones a la vista de contenedores y decisiones que el diagrama dejaba abiertas

Vigencia actual: registro histórico. ADR-16 sustituye su topología y revisa las relaciones de Pagos/datos; los mecanismos conservados se enumeran allí.
Estado: *aceptado; complementa a ADR-14*, no lo supera. El estilo híbrido, los catorce servicios

y el criterio de extracción no cambian. Lo que cambia son **cuatro relaciones** del diagrama de contenedores y **seis puntos** que ADR-14 no fijaba.

Contexto: la revisión del diagrama de contenedores de ADR-14 lo contrastó contra una especificación escrita a partir de él y encontró dos tipos de problema.

- **Relaciones que contradicen el propio diseño.** Acceso QR llamaba de forma síncrona a Ventas para gastar cada QR, aunque ADR-14 lo separa justamente porque su pico —la puerta— ocurre en otro momento que el de la venta: con esa llamada, una Ventas lenta atascaba la puerta. El *webhook* de la pasarela llegaba directo a Pagos, saltándose el punto único de entrada. Y el gateway le preguntaba a Autenticación por cada petición, lo que ponía a quien emite los tokens en el camino de todo el tráfico.
- **Decisiones sin tomar.** El diagrama no dice con qué se construyen los servicios, qué broker ni qué motor de búsqueda se usan, cuántas réplicas mínimas llevan los microservicios que «escalan solos», cómo se invalida el QR anterior en una reventa, cómo se avisa de un evento cancelado, ni cómo se garantiza que un cambio guardado llegue al broker.

Decisión — relaciones corregidas:

1. **El API Gateway valida el JWT localmente, con la clave pública**, y autoriza por el rol del token. No llama a Autenticación en cada petición: Autenticación solo emite tokens (*login*, renovación). Los servicios siguen verificando el token por su cuenta y comprobando en Redis que la sesión no esté cerrada (RNF-06, ADR-11).
2. **El gateway exige el turno de la Cola virtual en la ruta de compra** y lo gasta antes de reenviar la petición a Ventas. Ventas no llama a la Cola: solo recibe a cuantos puede atender. El turno es de un solo uso y solo vale para la cuenta que lo pidió.
3. **Acceso QR no llama a Ventas.** Mantiene su propia copia del estado de cada QR, alimentada por `CompraConfirmada` (activa los QR) y por un evento nuevo, `EntradaActualizada`, que Ventas publica cuando una entrada sale a reventa, vuelve, cambia de dueño o se anula. La validación pasa a ser **una transacción local** —QR, aforo y registro del ingreso— en lugar de una Saga con compensación remota. En sentido contrario, Acceso QR publica `IngresoRegistrado` y Ventas marca la entrada como usada: sin eso permitiría reembolsar (CU-003D) o revender una entrada con la que ya se entró.
4. **El *webhook* de la pasarela entra por el gateway**, por la única ruta pública sin JWT; Pagos lo autentica con la firma HMAC del cuerpo. Con él, Pagos publica `PagoAprobado` / `PagoRechazado`, y un cobro que quedó indeterminado por un *timeout* se resuelve cuando la pasarela confirma.

Decisión — lo que el diagrama dejaba abierto:

5. **Tecnologías.** Backend en **NestJS** (ya decidido en ADR-09); bases en **PostgreSQL** (ADR-06); broker **RabbitMQ** (ADR-10) con colas *quorum*, que son su forma de «tópicos replicados»; e índice de búsqueda sobre una **instancia PostgreSQL dedicada con búsqueda de texto completo** (`tsvector` con índice GIN y `pg_trgm` para errores de es-

critura). Se prefiere a OpenSearch porque los filtros de CU-005 —categoría, fecha, ciudad, artista— no necesitan relevancia ni facetas, y así el índice no añade una tecnología más que operar. Se revisa si la búsqueda empieza a necesitar ordenar por relevancia.

6. **Réplicas mínimas:** las de la vista física (§12), que el diagrama no repetía. Ventas **3**, igual que Cola virtual y Catálogo, porque los tres forman el camino de compra; Pagos y Autenticación **3**, porque si caen degradan muchos casos de uso a la vez; Acceso QR **2**, porque su carga se concentra en las horas de ingreso y escala por validaciones por segundo.

7. **Transactional Outbox** para todo evento que un servicio publique como consecuencia de escribir en su base: el evento se guarda en una tabla de salida *en la misma transacción* que el cambio, y un publicador lo saca al broker después. Se elige frente a «publicar después del commit» porque ese hueco pierde el evento si el broker o el proceso caen, y sin `CompraConfirmada` una entrada pagada no entraría por la puerta. La entrega es «al menos una vez»: cada evento lleva un `id` y una `secuencia` para descartar repetidos y desordenados.

8. `EventoCancelado`, publicado por Eventos y recintos y consumido por Catálogo (lo retira), Ventas (cierra la venta), Acceso QR (rechaza en la puerta) y Notificaciones (avisa a los compradores). Con solo `EventoPublicado`, cancelar no tenía cómo llegar a nadie.

Con esto el diagrama tiene **27 contenedores**: 4 *frontends*, el gateway, 7 microservicios, 7 servicios de dominio, 5 almacenes, el broker y 2 sistemas externos. La especificación de la revisión decía 34, pero su propia tabla suma 27.

Alternativas consideradas:

- **Mantener la llamada de Acceso QR a Ventas** y protegerla con un *circuit breaker*. Acota el daño pero no lo quita: con el circuito abierto la puerta deja de validar, que es justo lo que ADR-14 quería evitar.

- **Que Ventas gaste el turno llamando a la Cola.** Funciona, pero añade una dependencia síncrona más en el servicio que más carga soporta, y una flecha que el diagrama no tiene.

- **Validar el JWT en cada servicio y no en el gateway.** Ya se hace (RNF-06); hacerlo también en el gateway rechaza lo inválido antes de consumir un salto de red y una réplica.

Atributo de calidad: Disponibilidad —la puerta ya no depende de Ventas, ni el tráfico de Autenticación—; **Rendimiento**, porque una validación de QR y una autenticación dejan de costar un salto de red; **Consistencia**, por el Outbox y por la validación atómica en la puerta; y **Seguridad**, porque el turno solo vale para su dueño y el *webhook* solo se acepta firmado.

Consecuencias:

- **Consistencia eventual en la puerta.** Una entrada recién comprada tarda en poder entrar lo que tarda su evento en llegar —medio segundo en condiciones normales—. Para una puerta que abre horas después de la compra es irrelevante.

- **RNF-02 se cumple por orden, no por atomicidad.** Antes, el QR viejo y el nuevo

se cambiaban en el mismo `UPDATE` que consultaba la puerta. Ahora la puerta deja de aceptar la entrada cuando sale a reventa —mucho antes de transferirla—, así que el QR viejo ya no vale cuando llega el nuevo. Queda una ventana: entre que Ventas marca la entrada `EN_REVENTA` y la puerta se entera, el vendedor podría entrar con ella. Si después se transfiere, Acceso QR conserva el estado `USADA` y registra una alerta para resolverla con el comprador.

- **Ventas ya no sabe quién entró.** `USADA` vive solo en Acceso QR. No afecta a la cancelación ni a la reventa, que cierran antes de que empiece el evento.
- **Una sesión cerrada pasa el gateway** y la rechaza el servicio de destino. La garantía es la misma; cambia quién responde el 401.
- **Un turno se gasta aunque la reserva falle** —por ejemplo, si la localidad se agotó—. Es lo correcto para el aforo de la cola, pero obliga al comprador a volver a la fila si quiere intentar otra localidad.

Estado de implementación. Ya en el código: la validación local del JWT, la autorización por rol y la exigencia del turno en el gateway (`App/gateway/autenticacion.js`); la ruta pública del *webhook* con firma HMAC y la publicación de `PagoAprobado` / `PagoRechazado` en Pagos; el Outbox en Ventas para `CompraConfirmada` y `EntradaActualizada` y en Acceso QR para `IngresoRegistrado`; y la copia local de los QR en Acceso QR, que ya no llama a Ventas. Pendiente, porque depende de servicios que todavía no existen: `EventoPublicado` y `EventoCancelado` (Eventos y recintos), el consumo de `PagoAprobado` en Ventas para cerrar la Saga (fase E de `MIGRACION-ADR-14.md`), el Outbox en los demás publicadores, y el índice de búsqueda dedicado del Catálogo.

ADR-16 — Núcleo modular, Boletería, Operación y Trabajador; Pagos y datos compartidos

Estado: decisión acordada para la migración, implementada y ensayada en `feature/sebastian-arquitectura`; revisión por el equipo e integración a `develop` pendientes. No equivale a despliegue habitual. Fecha: 2/11/2026.

Contexto. El coste de coordinar programas, versiones, llamadas y datos superaba la separación necesaria para migrar lo ya construido. Las diez aplicaciones existentes y los catorce servicios propuestos por ADR-14 no coincidían. El objetivo es conservar comportamiento y dejar ubicaciones claras para que cada integrante termine sus ocho CU; no completar sus pendientes durante esta reorganización.

Decisión. Un workspace backend y cuatro responsabilidades de programa: Núcleo modular, Boletería, Operación y Trabajador. Existen los primeros dos y el consumidor de correo del Trabajador; Operación no tiene backend terminado. Pagos es módulo reutilizable, sin servidor propio; webhook único en Núcleo. Base PostgreSQL única por ocho esquemas y libro financiero único, preparación de datos coordinada antes de aplicaciones, conexiones nombradas y synchronize=false. Se conservan contratos públicos, autorización, migraciones y autoridades de

Entradas.

Sustituye. ADR-14 en granularidad/extracción y topología de catorce servicios; ADR-12 en la composición de su agregado; ADR-01 en servidor/base por dominio; ADR-06 en bases independientes/MongoDB como destino de esta migración. Revisa parcialmente ADR-15: llamadas internas a Pagos/Ventas pasan a interfaces locales y su webhook llega a Núcleo. Conserva gateway, RS256, roles, turno único, HMAC y los outbox/proyecciones existentes; no convierte sus garantías propuestas en funciones terminadas. ADR-02/03/05/07–11 conservan sus mecanismos aplicables. Los textos y diagramas anteriores quedan identificados como historia.

Concesiones y revisión de requisitos arquitectónicos.

- Se acepta despliegue/versionado/migraciones coordinados y un dominio de fallo SQL compartido; no independencia de base por microservicio. Los pools siguen separados y requieren dimensionado conjunto. No se afirma aislamiento de capacidad.
- RNF-10 conserva escalado independiente de Boletería frente al Núcleo; el escalado de cada componente de Entradas deja de ser individual. Requiere nueva medición, no reutilizar cifras de ADR-12/14 como cumplimiento.
- El aislamiento de proceso de Operación se conserva como objetivo para CU-010/019/020. La activación con Núcleo caído y alertas con acuse siguen pendientes.
- No se incorpora MongoDB, índice de búsqueda, Kubernetes o réplica de lectura para cerrar la migración. Sus propuestas/ASR anteriores requieren revisión del equipo; reportes y búsquedas ausentes no se declaran cumplidos con PostgreSQL vacío.
- Compartir SQL no aporta atomicidad entre operaciones existentes ni elimina ventanas QR/proyecciones. F12/F13/F14/F15 continúan abiertos. No se modifica el SRS, los umbrales RNF o el reparto para aparentar cumplimiento.

Alternativas. Conservar todos los servidores mantiene complejidad de red y versiones sin evidencia de necesidad. Un único backend reduciría más procesos, pero perdería el aislamiento previsto de Boletería/Operación. Una plataforma nueva de trabajos o reescribir todos los CU ampliaría el alcance sin preservar primero la referencia. Se elige migración incremental y pruebas reales sobre copias.

Verificación y límites. La evidencia local de `ESTADO-MIGRACION` apartado 23 cubre 830 unitarias y recorridos transferido/vacío, 73 Núcleo, 23 Boletería y contratos financieros/correo, imágenes y gateway. Las ejecuciones remotas, digests y bloqueos se registran en el apartado 24 y en la solicitud de integración, sin equiparar YAML válido a CI/CD ejecutado. Corte/reversión habitual están documentados, no ejecutados. Después de escrituras nuevas volver de imagen no revierte datos.

Consecuencia para el equipo. Los responsables del SRS conservan sus CU. Guía por integrante: `GUIA-EQUIPO-MIGRACION.md`. Completar los CU ausentes, F12 o evacuaciones no es requisito para terminar la migración del código existente.

15. Tácticas y mediciones históricas por atributo de calidad

Historia, no evidencia de ADR-16. Los resultados que siguen pertenecen a versiones anteriores. No se han repetido para certificar rendimiento/disponibilidad de la arquitectura actual. Sus guiones pueden alterar datos o pausar contenedores; usar las pruebas aisladas y consultar ESTADO-MIGRACION, sin ejecutarlos sobre el entorno habitual.

Las decisiones de la sección 14 dicen *qué* se eligió; esta sección dice **cómo se realiza** cada atributo priorizado en el código, con el nombre de la táctica, el mecanismo concreto que la implementa y la medición que la respalda. Cubre los cinco atributos que la entrega exige demostrar en vivo: **desempeño**, **disponibilidad**, **seguridad**, **consistencia** y **desplegabilidad**.

La columna **Evidencia** no es una promesa de diseño: son cifras obtenidas ejecutando los guiones de `pruebas/` contra el sistema desplegado, a través del API Gateway, que es por donde entra el tráfico real. Cada guion se puede volver a correr delante del evaluador.

Sobre qué arquitectura se midieron. Las cifras de esta sección se tomaron contra el sistema **desplegado**, que es el de ADR-12. Las tácticas en sí —balanceo, caché, reserva atómica, *circuit breaker*, idempotencia— son las mismas que ADR-14 conserva y amplía, pero los umbrales habrá que volver a medirlos cuando cambie la topología: un servicio que hoy atiende compra y consulta en el mismo proceso dará otros números cuando sean dos. Los ASR que ADR-14 añade —la cola virtual y la validación de QR sin red— todavía no tienen suite.

Las cinco mediciones se lanzan de una vez con `App/infra/atributos-calidad.py`, que corre cada suite y consolida sus cifras en `App/infra/informe-atributos/INFORME.md`. Las que aparecen en este documento son las de una corrida concreta; el guion da siempre las de la máquina donde se ejecute, y si una suite falla lo dice en vez de dar por buena una cifra que no se midió.

15.1. Desempeño

Escenario de referencia: la consulta de catálogo durante la apertura de venta (RNF-07, RNF-08), y el crecimiento de capacidad sin degradar otros dominios (RNF-10).

Tabla 10: Tácticas de desempeño, su mecanismo y la medición que las respalda.

Táctica	**Mecanismo en el sistema**	**RNF / ADR**	**Evidencia medida**
Mantener copias de los datos (caché)	Cabecera `Cache-Control: public` en la cartelera (30 s) y en el detalle de evento (300 s), de modo que navegador y CDN puedan responder sin llegar al servicio. Redis como almacén de acceso rápido para estado efímero.	RNF-07, ADR-03	La cifra de disponibilidad puede ir unos segundos por detrás; la compra vuelve a comprobar el cupo contra la base, así que la caché no puede sobrevender.
Introducir concurrencia	Varias instancias del servicio de Entradas tras el gateway, activables con `./iniciar.sh --replicas N`.	RNF-10, ADR-02	La cabecera `X-Servidor` devuelve la instancia que atendió cada petición, lo que hace visible el reparto.
Planificar recursos (balanceo)	Nginx resuelve el destino **en cada petición** (`resolver 127.0.0.11 valid=5s` más variable en `proxy_pass`) en vez de fijarlo al arrancar.	ADR-02	Una réplica nueva entra al reparto sin recargar el gateway.
Acotar la demanda de recursos	Pool de conexiones limitado por servicio (`POSTGRES_MAX_CONEXIONES`) y espera máxima acotada (`POSTGRES_ESPERA_MAXIMA_MS`).	RNF-07	Un pico de peticiones no agota las conexiones de la base ni deja sin atender al resto del servicio.
Resultado global (`rnf07-desempeno.py`, a través del gateway): p95 de **224,3 ms** con 200 peticiones simultáneas frente a un umbral de 500 ms; capacidad máxima observada **963 peticiones por segundo**; **0** peticiones fallidas.			

15.2. Disponibilidad

Escenario de referencia: el sistema sigue atendiendo ante la caída de una instancia o de una dependencia (RNF-03) y se recupera sin intervención manual (RNF-04).

Tabla 11: Tácticas de disponibilidad, su mecanismo y la medición que las respalda.

Táctica	**Mecanismo en el sistema**	**RNF / ADR**	**Evidencia medida**
Detección por eco (*ping/echo*)	Endpoint `/salud` que informa el estado **de cada dependencia** por separado, y `healthcheck` declarado en cada contenedor.	RNF-03	Respuesta observada con Redis caído: `{postgres: arriba, redis: caido}`.
Detección por temporizador	Tiempos máximos explícitos en cada salida: 2 s contra Redis (`commandTimeout`) y 10 s contra la pasarela de pagos.	RNF-03	Ninguna dependencia lenta puede bloquear indefinidamente una petición.
Fallar rápido	El cliente de Redis no encola peticiones cuando está caído (`enableOfflineQueue: false`, `maxRetriesPerRequest: 1`): devuelve error de inmediato.	RNF-03	El servicio responde 503 en lugar de acumular peticiones colgadas.
Degradación elegante	El inicio de sesión distingue las dependencias esenciales de las que no lo son: sigue operando sin Redis ni RabbitMQ, y solo rechaza si falta PostgreSQL.	RNF-03	Con Redis caído: **200 en 67 ms**. Con RabbitMQ caído: **200 en 67 ms**. Sin PostgreSQL: 503 explícito.
Reintento con cola de descartes	Los mensajes de cambio de turno se reintentan hasta `MAXIMO_INTENTOS = 3` y después pasan a una cola de mensajes muertos (*DLQ* nativa).	ADR-10	Un mensaje defectuoso no bloquea el procesamiento del resto de la cola.
Reintroducción automática	Política `restart: unless-stopped` en todos los contenedores del sistema.	RNF-04	Tras `kill -9`, el servicio vuelve a atender inicios de sesión en **1,3 s** (el requisito concede 30 s).
Resultado global (`cu027-disponibilidad.py`): **123 inicios de sesión por segundo** con 300 usuarios simultáneos; el sistema sobrevive a la caída de dos de sus tres dependencias y se recupera solo de la muerte del proceso.			

15.3. Seguridad

Escenario de referencia: toda petición a un microservicio llega autenticada y autorizada por rol desde el API Gateway (RNF-06), y el sistema nunca guarda datos completos de tarjeta (RNF-05).

Tabla 12: Tácticas de seguridad, su mecanismo y la medición que las respalda.

Táctica	**Mecanismo en el sistema**	**RNF / ADR**	**Evidencia medida**
Autenticar a los actores	Token JWT firmado con `RS256` por el Servicio de Administración con su clave privada. Los demás servicios solo llevan la **clave pública**: pueden verificar, no emitir. Dos tokens separados, de acceso y de renovación.	RNF-06, ADR-11	Se rechazan token firmado con otra clave, `alg: none`, `HS256` usando la clave pública como secreto, caducado, de otro emisor y manipulado: **6 de 6** responden 401.
Autorizar a los actores	Guarda `SesionValida` con `@RolesPermitidos` en cada controlador. El gateway valida el JWT con la clave pública y el rol de la ruta antes de enrutar (`autenticacion.js`, ADR-15) y propaga la identidad ya comprobada en `X-Usuario-Id` y `X-Usuario-Roles`.	RNF-06, ADR-02	**11 de 11** rutas de negocio responden 401 sin token (**0 % abiertas**). Un rol `Personal` recibe 403 en la reventa; el barrido de mantenimiento da 403 a `Cliente` y 200 a `Administrador`.
Limitar el punto de entrada	El gateway es la única puerta (ADR-02) y enumera las rutas una a una con coincidencia exacta (`location =`), de modo que una ruta nueva no queda expuesta por descuido.	RNF-06, ADR-02	La cartelera pública es la **única** excepción declarada, justificada en `DECISIONES.md` §12.

Táctica	**Mecanismo en el sistema**	**RNF / ADR**	**Evidencia medida**
Revocar el acceso	Lista de sesiones revocadas en Redis, compartida por todos los servicios. Cerrar sesión o cambiar la contraseña la alimenta al instante.	RNF-06, ADR-03	El mismo token, aún sin caducar, pasa a recibir 401 **al instante** tras cerrar sesión. Cambiar la contraseña cierra las demás sesiones y deja viva la que hizo el cambio. La revocación vive lo que le quede al token más un margen (**960 s** medidos).
Cerrarse ante la duda	Si Redis no responde, la comprobación de revocación no se puede hacer: se responde 503 en vez de aceptar un token que podría estar revocado.	RNF-06	Con Redis en pausa el acceso da **503**, no 200; al volver Redis, el mismo token vuelve a entrar sin intervención.
Proteger los datos almacenados	Las contraseñas se guardan con `scrypt` (sal por usuario y comparación en tiempo constante). De los pagos solo se guarda una referencia opaca de la pasarela: ningún dato de tarjeta entra al sistema.	RNF-05	Ni el esquema ni los eventos publicados tienen campo alguno para número de tarjeta; el cobro viaja como `token` del medio de pago.
Limitar lo que se expone	El DTO del mercado omite a propósito el `vendedorId`: quien compra necesita saber qué se vende y a qué precio, no la identidad de quien lo vende.	RNF-06	Comprobado en `cu006b-publicaciones.py`: el campo no aparece en ninguna respuesta del listado.
Resultado global (`rnf06-autenticacion.py`, con un inicio de sesión real contra Administración): **0 %** de rutas de negocio accesibles sin token; **9** formas distintas de token inválido, todas rechazadas; revocación efectiva en el acto; y ninguna aceptación de tokens cuando la comprobación no se puede realizar.			

15.4. Consistencia

Escenario de referencia: ninguna entrada publicada puede venderse a dos compradores (RNF-01) y, al transferirse, el código QR anterior queda invalidado antes de emitir el nuevo (RNF-02). Es el atributo que sostiene el caso de uso más complejo del sistema, el CU-006.

Tabla 13: Tácticas de consistencia, su mecanismo y la medición que las respalda.

Táctica	**Mecanismo en el sistema**	**RNF / ADR**	**Evidencia medida**
Mantener la coherencia con bloqueos	Bloqueo pesimista de PostgreSQL (`pessimistic_write`) sobre la fila de la compra o la publicación mientras se decide. Quien llega segundo espera, no lee un estado a medias.	RNF-01	Base del resultado global de abajo.
Escrituras condicionales	Los cambios de estado van como `UPDATE ... WHERE estado = <el esperado>` y **se mira cuántas filas cambiaron**: si son cero, otro llegó antes y se aborta. Es lo que decide el ingreso por QR (`VALIDA` → `USADA`) y la emisión de entradas.	RNF-01, RNF-02	Dos puertas escaneando el mismo QR a la vez: **un** 200 y **un** 409, **1** ingreso registrado y el aforo sube exactamente **1**.
Bloqueo distribuido con dueño	Reserva del checkout en Redis con `SET NX PX`; se libera con un guion Lua que solo borra la clave **si el testigo sigue siendo el suyo**. Antes de cobrar se renueva, y si la renovación falla se detiene el cobro.	RNF-01, ADR-03	50 compradores simultáneos sobre la misma entrada: **1** checkout abierto y **49** rechazados. Pagar con la reserva ya caducada da `CHECKOUT_NO_VIGENTE` sin llegar a cobrar.

Táctica	**Mecanismo en el sistema**	**RNF / ADR**	**Evidencia medida**
Claves de idempotencia	Cada cobro y cada reembolso llevan una clave estable hacia la pasarela, y los eventos publicados usan el identificador de la transacción como clave. Reintentar consulta el resultado, nunca vuelve a cobrar.	RNF-01, ADR-04	Reintentar el pago con la misma clave devuelve el **mismo** cobro y deja **una** sola transacción; con la clave de otro pedido se rechaza (`CLAVE_IDEMPOTENCIA_INCOMPATIBLE`).
Invariantes en la base de datos	Índice único parcial `uq_publicacion_activa_por_entrada` (una sola publicación activa por entrada), `UNIQUE` sobre `ingresos.entrada_id` y sobre el código QR, y restricciones `CHECK` que obligan a que el reparto cuadre (`precio = comision + neto_vendedor`) y a que una transacción aprobada tenga referencia.	RNF-01, RNF-11	El reparto cuadra al céntimo en todas las transacciones comprobadas. Aunque fallara la lógica, la base no admitiría el estado incoherente.
Una sola transacción para el cambio	La transferencia entera — propietario, código QR nuevo, cierre de la publicación e historial — va en una única transacción de base de datos. O cambia todo, o no cambia nada.	RNF-01, RNF-02	El QR anterior **no existe en ninguna fila** después de la transferencia; no hay estados intermedios que reparar.
Registro inmutable	Un disparador (`trg_historial_propietarios_inmutable`) rechaza `UPDATE` y `DELETE` sobre el historial de propietarios: solo admite inserciones.	RNF-11	La cadena de propiedad queda `EMISION` → `REVENTA` y no se puede reescribir desde la aplicación.
Ordenar para fallar sin daño	En la cancelación se anula primero y se reembolsa después: así la entrada no se puede usar en la puerta mientras el dinero está en vuelo. Si la pasarela rechaza, las entradas vuelven a ser válidas.	RNF-01	Reembolso rechazado (CU-003C): la entrada vuelve a `VALIDA`, la compra sigue `PAGADA` y el intento queda registrado como `RECHAZADA`.

Táctica	Mecanismo en el sistema	RNF / ADR	Evidencia medida
Resultado global (`rnf01-concurrencia.py`, tres escenarios sobre la misma entrada): **0** ventas duplicadas. Con 50 compradores a la vez, 1 checkout y 49 rechazos; con la reserva caducada, 1 cobro aprobado y 1 transferencia; con los dos pagando simultáneamente, **1** pago aceptado, **1** cobro y **1** QR emitido.			

15.5. Desplegabilidad

Escenario de referencia: desplegar una versión nueva de un microservicio sin interrumpir el servicio (RNF-15), y levantar el sistema completo — distribuido en dos o más computadores — desde un único guion.

Tabla 14: Tácticas de desplegabilidad, su mecanismo y la evidencia que las respalda.

Táctica	Mecanismo en el sistema	RNF / ADR	Evidencia medida
Desacoplar las unidades de despliegue	Una base de datos por microservicio, sin llaves foráneas entre dominios: las referencias cruzadas se resuelven en la aplicación. Cada servicio se despliega por separado.	RNF-13, RNF-15, ADR-01	Un cambio de esquema en un dominio no obliga a desplegar los demás.
Gestionar la tubería de despliegue	Integración continua de 8 trabajos (tipos, pruebas y compilación por servicio y por portal) y entrega continua que construye y publica las 6 imágenes desplegables.	RNF-15	La entrega continua incluye un trabajo que **baja lo publicado y comprueba que arranca**, antes de darlo por bueno.
Artefactos inmutables con vuelta atrás	Cada imagen se etiqueta con la rama y con el identificador exacto del commit (`sha-XXXXXXX`).	RNF-15	Volver a una versión anterior es desplegar su etiqueta, sin reconstruir ni tocar el repositorio.

Táctica	**Mecanismo en el sistema**	**RNF / ADR**	**Evidencia medida**
Empaquetado uniforme	Todos los componentes se distribuyen como contenedores, construidos para `linux/amd64` y `linux/arm64`.	ADR-01	La misma imagen corre en los servidores de integración y en los portátiles del equipo.
Automatizar el arranque	Un único guion (`App/infra/iniciar.sh`) levanta la infraestructura, aplica las migraciones pendientes, siembra los datos y prepara el inventario en memoria. Admite repartir el sistema en dos computadores (`--rol datos` y `--rol servicios --datos <IP>`).	RNF-15	De cero a sistema completo y utilizable en **2 min 10 s**, verificado borrando previamente contenedores y volúmenes.
Despliegue sin código fuente	La opción `--registro <etiqueta>` despliega desde las imágenes ya publicadas, y `--datos-demo sql` carga los datos sin necesitar Node ni el código.	RNF-15	Una máquina con solo Docker levanta el sistema con datos usables.

16. Riesgos técnicos

Riesgo	**Probabilidad**	**Impacto**	**Prioridad**	**Plan de mitigación**	**Plan de contingencia**
Concurrencia entre compradores en el mercado secundario	Media	Alto	Alta	Bloqueo distribuido con TTL en Redis antes de procesar el pago.	Rechazar la segunda transacción y notificar de inmediato al comprador no atendido.
Caída del API Gateway como punto único de entrada	Baja	Alto	Alta	Múltiples instancias + balanceador de carga + *health checks*.	Failover automático hacia una instancia o región de respaldo.

Riesgo	Probabilidad	Impacto	Prioridad	Plan de mitigación	Plan de contingencia
Indisponibilidad de la pasarela de pagos externa	Media	Alto	Alta	*Timeouts*, reintentos y cola de reconciliación de pagos.	Marcar la transacción como pendiente y permitir reintento manual del usuario.
Picos de tráfico en apertura de venta / ingreso masivo	Alta	Medio	Alta	Autoescalado del servicio de Entradas + caché en Redis.	Sala de espera virtual (cola de acceso) para limitar la concurrencia admitida.
Fallo del proveedor de notificaciones durante una emergencia	Baja	Alto	Media	Reintentos automáticos + canal secundario (SMS si falla push).	Escalar el aviso de forma manual mediante el personal en sitio.
Pérdida de conexión de una app móvil durante una evacuación	Media	Medio	Media	Caché local de instrucciones + reconexión automática.	El personal de seguridad releva la comunicación directamente en sitio.

17. Glosario

- **SAD:** Descripción de la Arquitectura del Software (*Software Architecture Description*), este documento.
- **ASR:** Requisito Arquitectónicamente Significativo (*Architecturally Significant Requirement*).
- **ADR:** Registro de Decisión Arquitectónica (*Architecture Decision Record*).
- **CU:** Caso de Uso.
- **QA:** Atributo de Calidad (*Quality Attribute*).
- **Modelo C4:** notación de Simon Brown para diagramar arquitectura en niveles (Contexto, Contenedores, Componentes, Código).
- **API Gateway:** componente que centraliza el enrutamiento, la autenticación y la autorización de las solicitudes al backend.

- **Microservicio:** servicio backend independiente, responsable de un dominio de negocio, con su propia base de datos.
- **Redis:** base de datos en memoria usada como caché y para bloqueos distribuidos.
- **RabbitMQ / Kafka:** sistemas de colas de mensajes usados para procesamiento asíncrono entre servicios.
- **AMQP:** protocolo de mensajería usado por colas como RabbitMQ.
- **QR:** código de respuesta rápida (*Quick Response*) usado para identificar entradas y validar pedidos.
- **Aforo:** cantidad de personas presentes o permitidas en un recinto o zona.
- **Mercado secundario:** reventa de entradas ya adquiridas entre usuarios del sistema.
- **CDN:** red de distribución de contenido (*Content Delivery Network*) usada para servir archivos estáticos cerca del usuario.
- **Balanceador de carga:** componente que distribuye solicitudes entre varias instancias de un servicio.
- **Bloqueo distribuido:** mecanismo (implementado aquí con Redis) que impide que dos procesos modifiquen simultáneamente el mismo recurso.
- **Consistencia eventual:** garantía de que los datos de distintos servicios convergen al mismo estado, aunque no de forma instantánea.

18. Referencias

- arc42. *arc42 overview.* `https://arc42.org/overview`
- Brown, S. *The C4 model for visualising software architecture.* `https://c4model.com`
- Brown, S. *Software architecture diagram review checklist.* `https://c4model.com/diagrams/checklist`
- Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley. (Base teórica del Árbol de Utilidad y los ADR usados en este documento.)
- Documentos internos del proyecto: *ArchitecturalProposal.tex*, *C4Diagrams.tex*, *Submission/DistribucionCasosUso.pdf* y *BitacoraArquitectonica.md* (equipo Hexacore, 2026).